

Solidity

Solidity는 객체 지향 high-level 언어로, 스마트 컨트랙트를 구현하기 위해 사용됩니다.

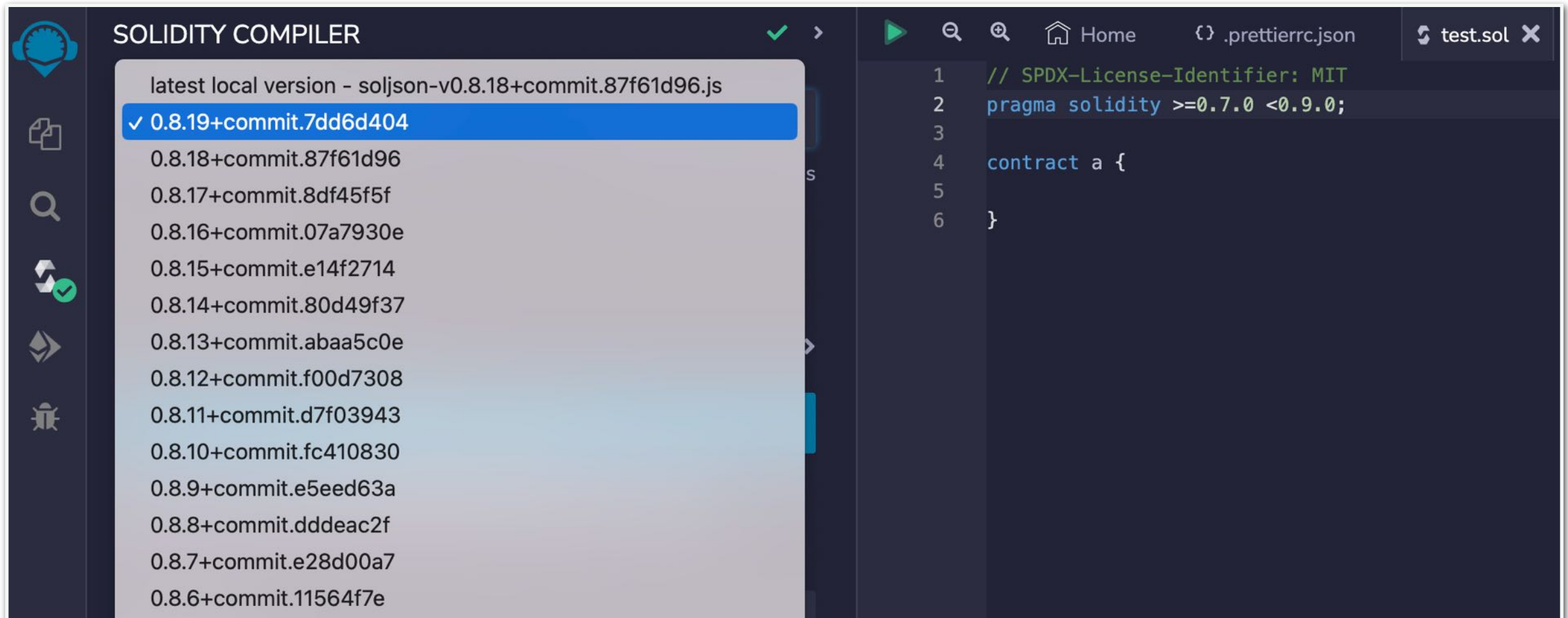
스마트 컨트랙트는 블록체인 네트워크에 저장 및 실행되는 프로그램이다. 계약 조건이 충족되면 미리 정의된 행동이 자동으로 수행된다.

<https://remix.ethereum.org>

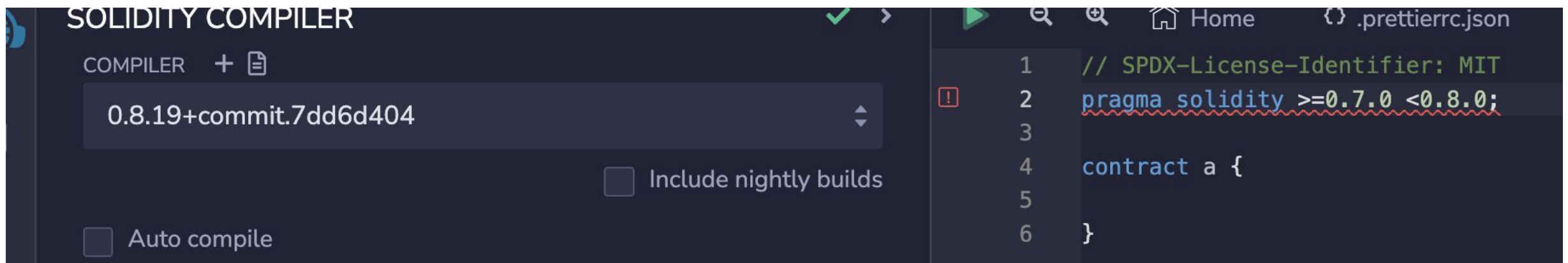
The screenshot displays the Remix Ethereum IDE interface. The browser address bar shows the URL `remix.ethereum.org/#lang=en&optimize=false&runs=200&evmVersion=null&version=soljson-v0.8.18+commit.87f61d96.js`. The interface is divided into several panels:

- FILE EXPLORER (Left Panel):** Shows the file structure of the `default_workspace`. It includes folders like `contracts`, `scripts`, `tests`, and `artifacts`, along with files like `README.txt`, `.prettierrc.json`, `test.sol`, `test1.sol`, and `test2.sol`. A yellow arrow points to the `contracts` folder. Two context menus are visible: one for "Create new file" and another for "Create new folder".
- Home Panel (Center):** Features the Remix logo and tagline "The Native IDE for Web3 Development." It includes links to [Website](#), [Documentation](#), [Remix Plugin](#), and [Remix Desktop](#). There is a "Search Documentation" input field. Below this, there are buttons for "New File" and "Open File", and a section for "Access File System" with options to load from `GitHub`, `Gist`, `IPFS`, and `HTTPS`.
- Learn Panel (Center):** Contains links to "Remix Basics", "Intro to Solidity", and "Deploying with Libraries".
- Featured Panel (Right):** Includes a "WATCH TO LEARN" section with a video titled "Video Tips from the Remix Team". Below this is a "Get Started - Project Templates" section with options for `GNOSIS SAFE MULTISIG`, `0XPROJECT ERC20`, and `OPENZEPPPELIN ERC20`. There is also a "Featured Plugins" section with `SOLIDITY`, `SOLHINT LINTER`, and `SOURCEIFY`.
- Scam Alert (Bottom Right):** A red-bordered box with a warning icon and text: "The only URL Remix uses is remix.ethereum.org. Beware of online videos promoting 'liquidity front runner bots': [Learn more](#). Additional safety tips: [here](#)".
- Bottom Panel:** Contains a search bar with the placeholder "Search with transaction hash or address" and a list of accessible libraries: `web3 version 1.5.2`, `ethers.js`, and `remix`. It also includes a checkbox for "listen on all transactions" and a prompt to "Type the library name to see available commands."

Solidity Compiler 버전



버전이 맞지 않으면, 아래와 같이 오류 발생



```

1  // SPDX-License-Identifier: GPL-3.0
2  pragma solidity ^0.8.7;
3
4  contract class00 {
5      // comment
6
7      /*
8         comments
9         comments
10      */
11  }

```

GPL : 무료로 사용이 가능하다. 해당 소스를 사용했으면 배포시 전체 소스코드를 공개해야한다.

컴파일러 버전은 0.8.7을 지원

contract 키워드는 스마트 컨트랙트 작성의 시작을 의미함

```

1  // SPDX-License-Identifier: MIT
2  pragma solidity >=0.7.0 <0.9.0;
3
4  contract class00 {
5      // comment
6
7      /*
8         comments
9         comments
10      */
11  }

```

MIT : 무료로 사용이 가능하고 아무런 제한이 없다. 무료 소프트웨어 라이선스이다.

컴파일러 버전은 0.7.0 부터 0.9.0 미만까지 지원

변수와 자료형

- 변하는 값을 갖는다.
- Value Type(값 타입)과 Reference Type(참조 타입)이 있다.
- Value Type: 값 자체가 복사됨
- Reference Type: 현재 해당하는 값의 메모리 주소가 복사
- Reference Type은 데이터의 저장위치를 명시해야함

자바스크립트 같은 동적 타입 언어와 달리 솔리디티는 정적 타입 언어이므로, 처음부터 변수를 선언할 때 자료형을 명확히 설정해야 한다

변수와 자료형

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity >=0.7.0 <0.9.0;
3
4  contract class00 {
5      // comment
6
7      uint8 a = 5;
8  }
```

uint8 a = 5

자료형 변수명 대입연산자 값

변수와 자료형

- address 타입: 20바이트 크기, 계정의 주소를 넣기 위해 사용
- bytes1~bytes32 타입: 1바이트 ~ 32바이트까지 넣을 수 있음
- string 타입: 문자열을 저장할 수 있고 Reference Type
- uint8~uint256: 양의 정수를 저장할 수 있으며 Value Type
- int8~int256: 정수를 저장할 수 있으며 Value Type

변수와 자료형

- 값 타입: uint, int, bool, address, bytes1, bytes2, bytes32
- 참조 타입: bytes, string, array, mapping, struct

변수와 자료형

uint8: $0 \sim 2^8-1$

int8: $-2^7 \sim 2^7-1$

uint8, uint16, uint32, uint64, uint128, uint256(=uint)

int8, int16, int32, int64, int128, int256(=int)

정수형

정수형을 int와 uint로 나누어서 사용

1. int: 부호가 있는 정수 자료형

- 예: int8, int16, int32, ..., int256
- 기본적으로 int는 int256을 의미합니다.
- 음수, 양수를 모두 표현 가능

2. uint: 부호가 없는 정수 자료형

- 예: uint8, uint16, uint32, ..., uint256
- 기본적으로 uint는 uint256을 의미합니다.
- 0과 양수만 표현 가능

```
contract IntegerExample {  
    int public negativeNum = -100;    // 부호 있는 정수  
    uint public positiveNum = 100;    // 부호 없는 정수  
}
```

주소형 (Address)

address: 이더리움 주소를 저장하는 자료형

address payable: 송금이 가능한(Payable) 이더리움 주소

(address에는 직접 이더를 전송할 수 없고, address payable이어야 transfer, send 같은 함수를 호출할 수 있습니다.)

주소형 변수는 20바이트(160비트) 크기를 가집니다.

```
contract AddressExample {
    address public owner;

    constructor() {
        owner = msg.sender; // 컨트랙트를 배포한 주소
    }

    function sendEther(address payable _to) public payable {
        // msg.value가 있어야 송금 가능
        _to.transfer(msg.value);
    }
}
```

바이트형 (Bytes)

고정 길이 바이트 배열과 동적 길이 바이트 배열

1. 고정 길이 바이트 배열: bytes1, bytes2, ..., bytes32
 - 정확한 길이를 지정해야 하며, 길이가 정해진 만큼만 저장 가능
 - 주로 해시값(예: 32바이트 짜리 bytes32)을 다룰 때 사용
2. 동적 길이 바이트 배열: bytes
 - 길이가 가변적
 - 하지만 가변적으로 저장하려면 추가 가스 비용이 듦

```
contract BytesExample {
    bytes32 public hashValue;

    // bytes는 동적 길이
    bytes public data;

    function setHash(bytes32 _hash) public {
        hashValue = _hash;
    }

    function setData(bytes memory _data) public {
        data = _data;
    }
}
```

문자열 (String)

문자열은 기본적으로 동적 길이로 string 자료형을 사용
내부적으로 bytes 형태로 관리

```
contract StringExample {  
    string public myString = "Hello Solidity";  
  
    function setString(string memory _newString) public {  
        myString = _newString;  
    }  
}
```

Boolean

bool: 참(true), 거짓(false)만 표현 가능

```
contract BooleanExample {  
    bool public isActive = true;  
  
    function toggleActive() public {  
        isActive = !isActive; // 현재 값의 반대로  
    }  
}
```

알아두기

1. 솔리디티에서 “어디에 데이터를 저장하느냐”도 변수 선언 시 중요함
 - a. storage: 블록체인 영구 저장소 (가스 비용이 큼)
 - b. memory: 함수 내 임시 저장소 (함수 호출 시에만 유효)
 - c. calldata: 주로 외부 함수 인자에서 사용, 변경 불가능(immutable)한 데이터
2. 함수 인수에 문자열이나 bytes 같은 동적 배열 자료형을 전달할 때는 memory 또는 calldata를 명시 필요
3. 타입 변환: uint ↔ int 변환 시 주의해야 함(음수 값 소실 가능).
4. 배열 다루기: 동적 배열과 정적 배열의 가스 사용량 다름.
5. 큰 정수 연산: 정수 오버플로(overflow) 문제 → 솔리디티 0.8.0 이상에서 자동으로 예외 처리.
6. 가시성(visibility): public, external, internal, private 함수나 변수에서 사용된 자료형의 범위와 스코프를 잘 이해해야 함.
7. 가스: 스마트 컨트랙트 실행에 필요한 연산 비용

연산자

산술 연산자: + - * / % **

논리 연산자: && ||

비교연산자: < > != == >= <=

연산자

DEPLOY & RUN TRANSACTIONS

ACCOUNT

0x5B3...eddC4 (99.9999999)

GAS LIMIT

3000000

VALUE

0 Wei

CONTRACT (Compiled by Remix)

class00 - class0418/class00.sol

Deploy

☐ Publish to IPFS

OR

At Address Load contract from Address

Transactions recorded 2

Deployed Contracts

CLASS00 AT 0XD91...39138 (MEMOR)

Balance: 0 ETH

assignment

Low level interactions

CALLDATA

Transact

Home class03_quiz.sol 1 class00.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity >=0.7.0 <0.9.0;
3
4 contract class00 {
5     // comment
6
7     string a = "Hello";
8
9     function assignment() public returns(string memory) {
10         a = "Hello Solidity!";
11         return(a);
12     }
13 }
14
```

listen on all transactions Search with transaction hash or address

✓ [vm] from: 0x5B3...eddC4 to: class00.assignment() 0xd91...39138 value: 0 wei data: 0x45f...65782 logs: 0 hash: 0x83f...9a64d

status	true Transaction mined and execution succeed
transaction hash	0x83f3d0b424d93aa7fed97e5335b8c9800eb5b8feb356cb775f49eedf2069a64d
from	0x5B38Da6a701c568545dCfcB03FcB875f56beddC4
to	class00.assignment() 0xd9145CCE52D386f254917e481eB44e9943F39138
gas	32368 gas
transaction cost	28146 gas
execution cost	7082 gas
input	0x45f...65782
decoded input	{}
decoded output	{ "0": "string: Hello Solidity!" }
logs	[]
val	0 wei

연산자

```
// SPDX-License-Identifier: MIT
pragma solidity >=0.7.0 <0.9.0;

contract class002 {
    uint a = 5+2;
    uint b = 5-2;
    uint c = 5*2;
    uint d = 5/5;
    uint e = 5%2;
    uint f = 5**2;

    function assignment() public view returns(uint,uint,uint,uint,uint,uint) {
        return(a,b,c,d,e,f);
    }
}
```

decoded output

```
{
  "0": "uint256: 7",
  "1": "uint256: 3",
  "2": "uint256: 10",
  "3": "uint256: 1",
  "4": "uint256: 1",
  "5": "uint256: 25"
}
```

연산자

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity >=0.7.0 <0.9.0;
3
4  contract class003 {
5      uint a = 5;
6      uint b = 5;
7      uint c = 5;
8      uint d = 5;
9      uint e = 5;
10
11     function assignment() public returns(uint,uint,uint,uint,uint) {
12         a += 2;
13         b -= 2;
14         c *= 2;
15         d /= 2;
16         e %= 2;
17         return(a,b,c,d,e);
18     }
19 }
```

```
decoded output      {
                    "0": "uint256: 7",
                    "1": "uint256: 3",
                    "2": "uint256: 10",
                    "3": "uint256: 2",
                    "4": "uint256: 1"
                    }
```

연산자

```
// SPDX-License-Identifier: MIT
pragma solidity >=0.7.0 <0.9.0;

contract class004 {
    uint a = 5;

    function justA() public view returns(uint) { 2415 gas
        return a;
    }

    function prePlus() public returns(uint) { infinite gas
        return ++a;
    }

    function postPlus() public returns(uint) { infinite gas
        return a++;
    }
}
```

return ++a

1

2

연산자 QUIZ

CLASS007QUIZ.SOL

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity >=0.7.0 < 0.9.0;
3
4  ✓ contract quiz1 {
5      |
6      |     uint public a = 4*2+1;
7      |     int public b = 2-1+3-5*2;
8      |     uint public c = 3*(3+3)/3;
9      |
10     | }
11
12  ✓ contract quiz2 {
13      |
14      |     bool public a = true||true||false;
15      |     bool public b = true&&true&&false;
16      |     bool public c = true||true&&false;
17     | }
18     |
```

상수

상수: 값이 명시된 이후 수정이 불가

변수: 값이 명시된 이후에도 수정 가능

```
uint constant a = 5;
```


상수

```
1  // SPDX-License-Identifier: GPL-3.0
2  pragma solidity >=0.7.0 < 0.9.0;
3
4  contract class008 {
5      uint constant a = 13;
6      string constant b = "hi";
7
8      function plusA() public pure returns(uint) {
9          |   return a+10; // 상수의 피연산자로 사용 가능할까?
10         }
11
12         function changeB() public {
13             |   b = "hello"; // 수정이 가능한가?
14         }
15     }
```

ENVIRONMENT

Remix VM (Merge)

VM

ACCOUNT

0x5B3...eddC4 (99.999999999998501916 ether)

GAS LIMIT

3000000

VALUE

0

Wei

CONTRACT (Compiled by Remix)

sm_a - class0418/test.sol

Deploy

☐ Publish to IPFS

OR

At Address Load contract from Address

Transactions recorded 3

Deployed Contracts

A AT 0XD91...39138 (MEMORY)

Balance: 0 ETH

Low level interactions

CALLDATA

Transact

Home test.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity >=0.7.0 <0.9.0;
3
4 contract sm_a {
5
6     uint public a = 1;
7     int public b = -1;
8     bool public c = true; // or false
9     bytes1 public d = hex"11";
10    string public e = "abcde";
11
12    //산술 연산자 : +,-,*,/
13    uint public f = 1+2;
14    uint public g = 3-2;
15    uint public h = 4*2;
16    uint public i = 4/2;
17    uint public j = 4**2;
18
19    //논리 연산자 : &&, ||
20    bool public k = true && true;
21    bool public m = true && false;
22    bool public n = false && false;
23
24    bool public l = true || true;
25    bool public o = true || false;
26    bool public p = false || false;
27
28    // 비교연산자
29    bool public q = 3>5;
30    bool public r = 3<=4;
31    bool public s = 3==3;
32    bool public t = 3!=3;
33
34
35    uint public constant z=3;
36 }
37
```


가시성 지정자

public : 내부, 외부에서 접근이 가능함

external : 외부 접근만 가능함 (함수에 사용, 변수에 적용 불가)

(예외적으로 this라는 키워드를 사용해서 접근 가능함)

private: 내부 접근만 가능함

internal: 내부 접근만 가능하나, 상속을 받은 경우 가능함

(private의 확장형이라고 생각하면 좋다)

가시성 지정자

```
uint public a = 5;
```

자료형 가시성 지정자 변수 대입연산자 값

```
uint b = 5;
```

(internal로 처리됨)

자료형 변수 대입연산자 값

함수

특별한 작업을 수행하도록 만들어진 코드의 집합체

function 키워드를 사용해서 정의한다.

function 함수명() public {

함수의 내용...

}

```
1  // SPDX-License-Identifier: GPL-3.0
2  pragma solidity >=0.7.0 <0.9.0;
3
4  contract class009 {
5
6      uint public a = 3;
7      function myFun() public{      22258 gas
8          a = 5;
9      }
10 }
```

함수

```
1  // SPDX-License-Identifier: GPL-3.0
2  pragma solidity >=0.7.0 <0.9.0;
3
4  contract class010 {
5
6      uint public a = 3;
7      function myFun(uint b) public{
8          a = b;
9      }
10
11 }
```

Deployed Contracts

▼ CLASS010 AT 0X7B9...B6ACE (MEMO)

Balance: 0 ETH

myFun

uint256 b

a

0: uint256: 3

Deployed Contracts

▼ CLASS010 AT 0XD4F...2CBEE (MEMO)

Balance: 0 ETH

myFun

333

a

0: uint256: 333

함수

```
1  // SPDX-License-Identifier: GPL-3.0
2  pragma solidity >=0.7.0 <0.9.0;
3
4  contract class011 {
5
6      uint public a = 3;
7      function myFun(uint b, uint c, uint d) public{
8          a = b;
9          a = c;
10         a = d;
11     }
12
13 }
```

Deployed Contracts

▼ CLASS011 AT 0X332...D4B6D (MEMO)

Balance: 0 ETH

myFun 1,2,3

a

0: uint256: 3

myFun

b: "11"

c: "22"

d: "33"

Calldata

Parameters

transact

a

0: uint256: 33

함수

```
function myFun() public returns (uint) {  
    // 함수 로직  
  
    return a  
    uint 자료형을 가진 변수 a  
}
```

함수

```
1 // SPDX-License-Identifier: GPL-3.0
2 pragma solidity >=0.7.0 <0.9.0;
3
4 contract class012 {
5
6     uint public a = 3;
7     function myFun() public returns(uint){
8         a = 100;
9         return a;
10    }
11 }
```

Deployed Contracts

▼ CLASS012 AT 0X5A8...C4D01 (MEMO)

Balance: 0 ETH

myFun

a

a

a - call

0: uint256: 100

to	class012.myFun() 0x5A86858aA3b595FD6663c2296741eF4cd8BC4d01
gas	27273 gas
transaction cost	23715 gas
execution cost	2651 gas
input	0x75b...bb4e7
decoded input	{}
decoded output	{ "0": "uint256: 100" }
logs	[]
val	0 wei

함수

```
1 // SPDX-License-Identifier: GPL-3.0
2 pragma solidity >=0.7.0 <0.9.0;
3
4 contract class013 {
5
6     uint public a = 3;
7     uint public b = 5;
8     function myFun() public returns(uint, uint){
9         a = 100;
10        b = 0;
11        return (a,b);
12    }
13
14 }
```

Deployed Contracts 

▼ CLASS013 AT 0X540...C7569 (MEMO)  

Balance: 0 ETH

myFun

a

0: uint256: 3

b

0: uint256: 5

Deployed Contracts 

▼ CLASS013 AT 0X540...C7569 (MEMO)  

Balance: 0 ETH

myFun

a

0: uint256: 100

b

0: uint256: 0

함수

```
1  // SPDX-License-Identifier: GPL-3.0
2  pragma solidity >=0.7.0 <0.9.0;
3
4  contract class014 {
5
6      function myFun() public pure returns(uint age, uint weight) {
7          age = 31;
8          weight = 60;
9      }
10 }
```

별도의 변수 선언이 없다.

별도의 리턴 키워드가 없다.

return (age, weight); 문장은 생략 가능하다.

함수

```
1  // SPDX-License-Identifier: GPL-3.0
2  pragma solidity >=0.7.0 <0.9.0;
3
4  contract class015 {
5
6      uint public a = 3;
7      uint public b = myFun();
8      function myFun() public returns(uint){
9          a = 100; // 추가 필요
10         return a;
11     }
12
13 }
```

Deployed Contracts

▼ CLASS015 AT 0XE5F...78E22 (MEMOF



Balance: 0 ETH

myFun

a

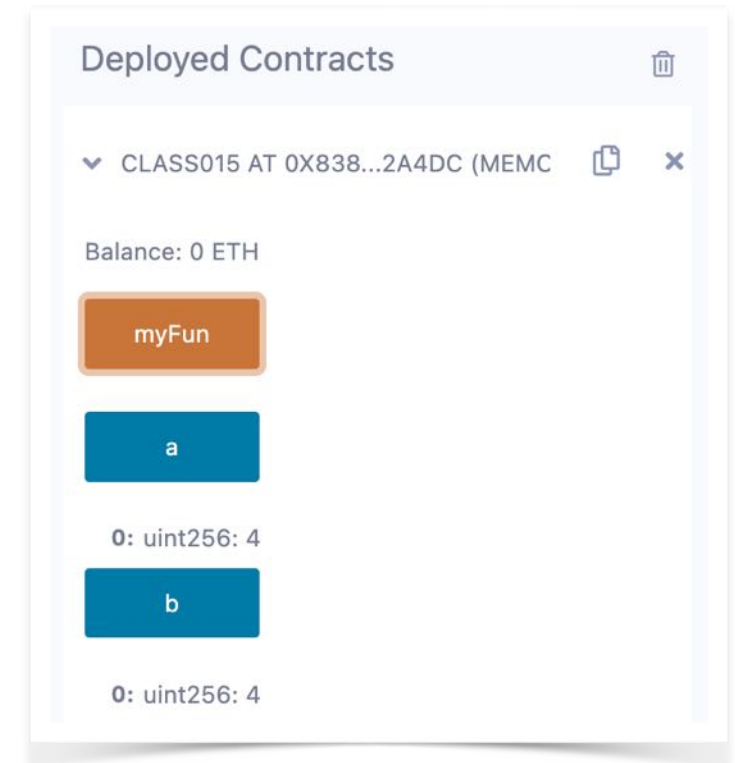
0: uint256: 100

b

0: uint256: 100

함수

```
1  // SPDX-License-Identifier: GPL-3.0
2  pragma solidity >=0.7.0 <0.9.0;
3
4  contract class015 {
5
6      uint public a = 3;
7      uint public b = myFun();
8      function myFun() public returns(uint){
9          a = a+1;
10         return a;
11     }
12
13 }
```



- b의 값이 정해지는 시점은?
- 버튼 b를 클릭하면 어떠한 작업이 수행되는가?

함수

```
1  // SPDX-License-Identifier: GPL-3.0
2  pragma solidity >=0.7.0 <0.9.0;
3
4  ✓ contract class016 {
5      |
6      |     uint public a = 3;
7      |     ✓ function myFun(uint b) public returns(uint){
8      |         |     a = b;
9      |         |     return a;
10     |     }
11     |
12 }
```

매개변수와 반환값이 있는 경우

State Mutability

pure: 함수 내부에 정의된 변수와 매개변수만 사용한다.

view: 외부 변수를 읽을 수 있음 + pure 속성

State Mutability

- pure 가 있는 함수
- view 가 있는 함수
- State Mutability(상태 가변성 지정자) 이 없는 함수

```
function 함수이름() public pure {  
  
}
```

함수

```
1  // SPDX-License-Identifier: GPL-3.0
2  pragma solidity >=0.7.0 <0.9.0;
3
4  contract class017 {
5      function myFun(uint a) public pure returns(uint){
6          return a;
7      }
8  }
```


함수

```
1  // SPDX-License-Identifier: GPL-3.0
2  pragma solidity >=0.7.0 <0.9.0;
3
4  contract class018 {
5
6      uint a = 3;
7      function myFun() public pure returns(uint){
8          a = 4;
9          return a;
10     }
11 }
```


함수

```
1  // SPDX-License-Identifier: GPL-3.0
2  pragma solidity >=0.7.0 <0.9.0;
3
4  contract class019 {
5      uint public a = 4;
6      function myFun() public view returns(uint){
7          uint b = a + 5;
8          return b;
9      }
10 }
```

함수

```
1  // SPDX-License-Identifier: GPL-3.0
2  pragma solidity >=0.7.0 <0.9.0;
3
4  contract class019 {
5      uint public a = 4;
6      function myFun() public view returns(uint){
7          a = a + 5;
8          return a;
9      }
10 }
11
```

함수

```
1  // SPDX-License-Identifier: GPL-3.0
2  pragma solidity >=0.7.0 <0.9.0;
3
4  contract class020 {
5
6      uint public a = 3;
7      function myFun() public view returns(uint){
8          a = 4;
9          return a;
10     }
11 }
```

참조 타입

참조타입이 함수의 매개변수, 반환값 또는 함수 내부의 변수로 정의될 때 저장 공간을 명시해야한다.

(Value Type: 값 자체, Reference Type: 값이 있는 곳 가르킴)

Value Type: 종이 쪽지를 복사하는 것 (원본과 복사본은 따로 존재)

Reference Type: 공유 문서 링크를 보내는 것 (링크를 통해 같은 문서를 본다)

대표적으로 string, bytes, array, struct, mapping이 있다.

참조타입을 사용할때는 저장위치를 명확히 지정해야한다.

저장공간: storage, memory, calldata

Reference Type

참조타입이 함수의 매개변수, 반환값 또는 함수 내부의 변수로 정의 될 때 저장 공간을 명시해야 한다.

직접 값을 가지지 않고 값이 어디에 저장돼 있는지만 가리키는 타입이다. 예를 들면 `string`, `bytes`, 배열(`array`), 구조체(`struct`)들 이다. 즉, "값"이 아니라 "위치"를 다루는 자료형이다. 그 저장 위치가 어디인지(`storage`, `memory`, `calldata`) 명확히 정해줘야 한다.

구분	필요한 이유
함수 매개변수	외부 입력값이니까 어디 저장할지 지정해야 함 (보통 <code>calldata</code>)
함수 반환값	어디에 있는 데이터를 돌려줄지 명확히 해야 함 (보통 <code>memory</code>)
함수 내부 변수	복사본을 어디다 둘 건지 정해야 함 (보통 <code>memory</code>)

Reference Type

참조타입이 함수의 매개변수, 반환값 또는 함수 내부의 변수로 정의 될 때 저장 공간을 명시해야 한다.

위치	예시	저장공간 명시 필요 여부	이유
함수 매개변수	function foo(string calldata s)	✓ 필요	외부에서 전달된 데이터를 어디 둘지 명시해야 함
함수 반환값	returns (string memory)	✓ 필요	어떤 저장소에서 리턴할지 명시 필요
함수 내부 변수	string memory temp = ...	✓ 필요	복사본의 위치를 정해야 함
상태 변수	string public name;	✗ 필요 없음	자동으로 storage 로 설정됨

Reference Type

```
contract ex_ref {  
  
    // 저장 공간 명시 안 해도 됨 (자동으로 storage로 설정됨)  
    string public name = "Hello";  
  
    // 함수 매개변수  
    function setName(string calldata _name) external {  
        ...  
    }  
  
    // 함수 반환값  
    function getName() external view returns (string memory) {  
        return name;  
    }  
  
    // 함수 내부 변수  
    function copyName() public view returns (string memory) {  
        string memory temp = name;  
        return temp;  
    }  
}
```


Reference Type

storage: 블록체인 상에 영구 저장(비용이 비쌈)

memory: 함수 호출시 생성되는 임시 저장소(함수가 리턴되면 사라짐)

calldata: 외부 함수 호출에서만 사용, 읽기 전용(변경 불가)

저장소	설명	수정 가능 여부	주 사용처
calldata	외부 입력 데이터 저장	✗ (읽기만 가능)	외부 입력값 처리 (ex: 문자열, 배열)
memory	일시적 복사본 저장	✓	내부 로직 처리
storage	블록체인에 영구 저장	✓	상태변수 저장

Value Type 리턴

값 타입은 리턴할 때 별도로 memory 같은 데이터 위치(location)를 지정할 필요가 없음
그냥 타입만 쓰면 됨: returns (uint), returns (bool), returns (address)

```
pragma solidity ^0.8.0;

contract Example {

    function getNumber() external pure returns (uint) {
        return 123;
    }

    function getBoolean() external pure returns (bool) {
        return true;
    }

    function getAddress() external view returns (address) {
        return msg.sender;
    }
}
```

Reference Type 리턴

참조 타입은 반드시 memory를 지정해야 함. 참조 타입은 값이 아니라 '위치'를 가리키는 자료형이므로, 컴파일러는 "이 데이터가 어디에 있는 건지"를 정확히 알아야만 외부로 리턴할 수 있습니다. 보통 리턴하는 데이터는 memory에 올려서 가볍게 외부로 보낸다고 이해하시면 됩니다. 거의 대부분 Reference Type 반환은 memory로 한다. (storage, calldata 못씀)

```
pragma solidity ^0.8.0;

contract Example {

    string public storedName = "Hello";

    function getName() external view returns (string memory) {
        return storedName;
    }

    function getBytes() external pure returns (bytes memory) {
        return "hello";
    }

    function getArray() external pure returns (uint[] memory) {
        uint ;
        arr[0] = 1;
        arr[1] = 2;
        arr[2] = 3;
        return arr;
    }
}
```

질문

답변

값 타입 리턴할 때 memory 써야 하나?

✗ 필요 없음

참조 타입 리턴할 때 memory 써야 하나?

✓ 반드시 써야 함

memory 대신 storage/calldata 쓸 수 있나?

✗ 보통 불가능 (memory만 사용)

조건문, 반복문

C언어 동일함!

예제코드 4장

mapping: Key, Value 형태 저장

선언

```
mapping(address=>uint) public balance;
```

값의 형태가 uint인 경우 기본값은 0으로 저장된다.

값 할당

```
balance[키] = 값;
```

삭제

```
balance[키] = 0;  
delete(balance[키]);
```

mapping

key를 통해 value를 저장하는 dictionary 구조.

```
mapping(KeyType => ValueType) public myMap;
```

선언

```
mapping(address => uint) public balances;
```

조회

```
balances[addr];
```

mapping

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity ^0.8.0;

contract TokenExample {

    // 주소 → 잔액
    mapping(address => uint) public balances;

    // 잔액 조회
    function getBalance(address _user) public view returns (uint) {
        return balances[_user];
    }
}
```


mapping

항목	설명
기본값 존재	값을 넣지 않아도 기본값(예: <code>0</code>)이 존재함
키 목록 조회 불가	저장된 키 전체를 조회할 수 없음
반복 불가	for문으로 순회 불가능
삭제는 가능	<code>delete balances[_addr];</code> 하면 값이 0으로 초기화됨

mapping

```
contract BirthdayBook {  
    mapping(string => string) public birthdays;  
  
    function setBirthday(string memory _name, string memory _birthday) public {  
  
    }    birthdays[_name] = _birthday;  
  
    function getBirthday(string memory _name) public view returns (string memory) {  
        return birthdays[_name];  
    }  
}
```

배열: Index, value 형태 저장

선언

```
uint[ ] public array1;  
uint[5] public array2 = [2, 11, 34, 33, 22];
```

동적 크기 배열: 크기를 자유롭게 늘리거나 줄일 수 있다.

정적 크기 배열: 크기가 고정되어 있다.

usage

```
array1.length  
array1.push(값)  
array1.pop()  
delete array[인덱스]
```

Array

1. 정적(고정) 크기 배열
2. 동적(가변) 크기 배열

```
uint[ ] public array1;  
uint[5] public array2 = [2, 11, 34, 33, 22];
```

사용예

```
contract FixedArray {  
    uint[3] public numbers = [1, 2, 3]; // 크기가 3으로 고정됨  
  
    function get(uint index) public view returns (uint) {  
        return numbers[index]; // 인덱스로 접근  
    }  
}
```

Array

```
contract DynamicArray {
    uint[] public values;

    function add(uint _val) public {
        values.push(_val); // 배열 끝에 추가
    }

    function get(uint _index) public view returns (uint) {
        return values[_index]; // 특정 위치 값 반환
    }

    function removeLast() public {
        values.pop(); // 마지막 값 제거
    }

    function length() public view returns (uint) {
        return values.length; // 배열 길이 반환
    }
}
```

Array

함수/키워드	설명	사용 예시
push()	끝에 값 추가 (가변 배열만 가능)	values.push(5);
pop()	마지막 값 제거	values.pop();
.length	배열 길이 반환	values.length
인덱스 접근	특정 위치 값 가져오기/수정	values[0] = 100;

항목	설명
인덱스 초과 접근 불가	존재하지 않는 인덱스를 접근하면 오류 발생
pop은 비어있는 배열에 쓰면 오류	pop()은 최소 1개 이상 있어야 작동
고정 배열은 push 불가	uint[3] 배열은 크기 초과시 값 추가 못 함
배열 전체를 리턴할 때는 memory 지정 필요	returns (uint[] memory)로 명시해야 함

솔리디티에서 배열을 `uint[] public arr;` 이라고 선언하고 이 배열에 값을 넣는 방법, 빼는 방법, 초기화 하는 방법은 다음과 같습니다.

값을 넣는 방법:

```
arr.push(5); // arr 배열에 5를 추가합니다.
```

값을 빼는 방법:

```
arr.pop(); // arr 배열의 마지막 요소를 제거합니다.
```

배열을 초기화 하는 방법:

```
delete arr; // arr 배열을 초기화합니다.
```

위의 각 코드는 `arr` 배열에 5를 추가하거나, 마지막 요소를 제거하거나, 배열을 완전히 초기화하는 데 사용됩니다.

Struct 사용예

Array와 Struct의 사용

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity ^0.8.0;

contract StudentRegistry {

    // 1. 구조체 정의
    struct Student {
        string name;
        uint age;
        bool isRegistered;
    }

    // 2. 구조체 배열 (여러 명 저장)
    Student[] public students;

    // 3. 학생 등록
    function registerStudent(string memory _name, uint _age) public {
        students.push(Student(_name, _age, true));
    }

    // 4. 학생 정보 조회
    function getStudent(uint _index) public view returns (string memory, uint, bool) {
        Student memory s = students[_index];
        return (s.name, s.age, s.isRegistered);
    }
}
```


Struct 사용예

Mapping과 Struct의 사용

```
contract StudentMap {  
  
    struct Student {  
        string name;  
        uint age;  
    }  
  
    mapping(address => Student) public studentMap;  
  
    function register(string memory _name, uint _age) public {  
        studentMap[msg.sender] = Student(_name, _age);  
    }  
  
    function get() public view returns (string memory, uint) {  
        Student memory s = studentMap[msg.sender];  
        return (s.name, s.age);  
    }  
}
```


문자열 비교

keccak256(바이트값) 함수는 매개변수 바이트값의 해시를 구하는 함수이다.

```
if ( keccak256(bytes(문자열변수)) == keccak256(bytes(문자열변수))) {  
  
}  

```

QUIZ 4, 5

데이터 저장공간에 따른 참조타입 저장 방식

관계	저장 방식	해설
memory와 memory	주소 복사	memory에 저장된 변수를 다른 memory 변수에 대입 시 값을 가리키는 주소를 공유해서 서로에게 영향을 줄 수 있다.
storage와 local storage	주소 복사	storage에 저장된 변수를 local storage 변수에 넣으면 값을 가리키는 주소를 공유해서 서로에게 영향을 줄 수 있다.
storage와 memory	값 복사	storage에 저장된 변수를 memory 변수에 넣으면 값을 복사하므로 서로에게 영향을 주지 않는다.
그 외의 storage가 포함된 관계	값 복사	앞에서 언급한 관계를 제외한 storage가 포함된 관계는 값을 복사한다. 예를 들어 storage와 storage의 경우가 있다.

event

event는 스마트컨트랙트가 외부(프론트엔드 등)에 정보를 알리기 위해 블록체인의 로그 영역에 기록하는 메시지 (영구히 기록되고 추적 가능)

목적	설명
외부 알림용	프론트엔드에서 스마트컨트랙트의 실행을 감지
트랜잭션 기록용	누가 무엇을 했는지 추적하는 블록체인 로그
디버깅용	트랜잭션 실행 시 어떤 값이 emit 되었는지 확인 가능
효율성	상태변수보다 가스비가 저렴함

event

```
// 1. 이벤트 정의  
event 이벤트이름(타입 변수1, 타입 변수2);
```

```
// 2. 이벤트 발생  
emit 이벤트이름(값1, 값2);
```

```
// SPDX-License-Identifier: MIT  
pragma solidity ^0.8.0;  
  
contract Greeting {  
  
    // ① 이벤트 선언: 누가(msg.sender), 어떤 인사(message)를 남겼는지 기록  
    event Greeted(address indexed sender, string message);  
  
    // ② 함수 내에서 이벤트 사용 (emit으로 기록)  
    // emit Greeted(msg.sender, "안녕하세요!");  
    function sayHello(string memory _message) public {  
        emit Greeted(msg.sender, _message);  
    }  
}
```

event

컨트랙트 안에서는 과거 emit된 이벤트 내용을 읽어올 수 없습니다.

- 상태변수는 `balances[msg.sender]`처럼 접근 가능
- 하지만 이벤트는 아래처럼 기록만 가능하고,

```
emit Voted(msg.sender, candidate, 1);
```

→ Voted 이벤트를 읽거나 다시 불러오는 문법은 없습니다.

즉, **블록체인 외부(예: Etherscan, 프론트엔드 앱)**에서는 로그를 읽을 수 있지만, 스마트컨트랙트 내부에서는 읽는 기능이 없습니다.

event: 간단한 투표 시스템

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity ^0.8.0;

contract Voting {

    mapping(string => uint) public voteCounts;
    mapping(address => bool) public hasVoted;

    // 이벤트 정의
    event Voted(address indexed voter, string indexed candidate, uint count);

    // 투표 함수
    function vote(string memory _name) public {
        require(!hasVoted[msg.sender], "이미 투표하셨습니다.");

        voteCounts[_name] += 1;
        hasVoted[msg.sender] = true;

        emit Voted(msg.sender, _name, 1); // 이벤트 발생
    }
}
```


Storage 변수와 event 차이

항목	상태변수 (Storage)	이벤트 (Event Log)
저장 위치	블록체인의 Storage 영역	블록내 트랜잭션의 Receipt → Logs 영역
읽기	컨트랙트 코드에서 읽기 가능	컨트랙트 코드 내부에서 읽기 불가능
쓰기	언제든지 가능 (a = 10)	emit으로 로그 작성만 가능
변경 가능 여부	수정 가능	수정 불가 (불변 기록)
목적	상태 관리 (현재값 추적)	실행 기록 남기기 (과거 기록 추적)

event

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract StateVsEvent {
    // 상태변수: 블록체인 저장소에 저장됨
    uint public stateCounter;

    // 이벤트: emit으로 기록만 가능, 다시 불러올 수 없음
    event CounterIncreased(address indexed sender, uint newValue);

    // 상태변수 증가 & 이벤트 기록
    function increase() public {
        stateCounter += 1; // 상태변수 값 증가
        emit CounterIncreased(msg.sender, stateCounter); // 이벤트 기록
    }

    // 상태변수 확인
    function getStateCounter() public view returns (uint) {
        return stateCounter;
    }

    // 아래는 "이벤트"를 읽는 함수처럼 보이지만 불가능함
    function getLastEvent() public view returns (???) {
        // 불가능: 이벤트는 컨트랙트 내부에서 다시 불러올 수 없음
    }
}
```

- 이벤트(Event)

- 스마트 컨트랙트 또는 유저의 특정한 상태를 출력하며 그 상태를 블록체인에 저장한다.

- 대표적인 이벤트는 Transfer 이벤트가 있다. from, to, amount로 구성되어 있다.

- storage에 저장하는 것 보다 event로 저장하는 것이 가스비용이 적다.

- Storage에 저장된 변수는 다른 값으로 변경할 수 있다. 그러나 이벤트에 기록된 값은 수정이 불가능하다.

- 이벤트는 블록체인에 영속적으로 기록이 된다.

- 이벤트를 기록하는 함수는 pure, view를 적용하지 않는다.

이벤트 정의 방법

```
event MyEvent(unit result, string name)
```

이벤트 출력

```
emit MyEvent(10, "add")
```

6-1

- 생성자

- 생성자는 스마트 컨트랙트가 배포될때 제일 먼저 실행되는 함수

- 생성자는 필수요소가 아니다

- 생성자는 한 개의 생성자만 가능
하다

```
contract Test {  
    constructor() {  
        ...  
    }  
}
```

6-2, 6-3

- 객체지향 관점에서 contract, object, instance

- contract는 class와 동일한 개념
- object는 자료형으로 선언된 변수이며 아직 값이 할당되지 않은 상태

```
Test public test1;
```

- object에 값이 할당되면 인스턴스라고 한다. 이더리움에서는 인스턴스화했다는 것은 스마트 컨트랙트를 새롭게 배포한 것과 같다.

```
Test public test1;
```

```
test1 = new Test()
```

Solidity에서 new를 하면 블록체인에 실제로 또 하나의 계약(컨트랙트)을 새로 배포하는 것이고, 그렇게 만들어진 인스턴스는 각자 고유한 주소와 독립적인 데이터 상태를 갖습니다.

6-4
name function

6-5
inheritance

6-8
encapsulation

- constant와 immutable의 큰 차이점은 초기값 선언 여부이다. 상수의 경우 초기값을 필수적으로 입력해야하지만, immutable은 초기값을 선언하지 않아도 된다. 그래서 immutable이 적용된 변수의 생성자에서 값을 입력할 수 있다. immutable은 오로지 값타입만 적용 할 수 있다. 결론적으로 상수를 선언할 때는 반드시 초기값을 세팅해야한다.

- 'is' 키워드를 통해 상속 관계를 정의한다.

- 오버로딩(다른 자료형의 함수 파라미터)과 오버라이딩(자식에서 재정의하는 것으로 virtual, override 키워드 사용)을 모두 지원한다.

- assert / revert / require: 특정 조건에 부합하지 않아 트랜잭션을 실패로 만들어야 할때는 오류를 발생시킨다.

- modifier: 다수의 함수에 특정 로직을 부여할 수 있다. '_'의 키워드는 모디파이어가 적용될 함수의 로직이 시작되는 지점을 나타낸다.

- assert, revert, require

- 트랜잭션을 실패로 만들때 사용한다. (컨트랙트 내부에서 이루어진 모든 변경 사항이 되돌려진다)
- 특정 조건이 부합하지 않아 트랜잭션을 실패로 만들어야 할때 오류를 발생시켜야 한다. assert, revert, require를 통해 오류를 발생 시킬 수 있다.
- assert(false), revert("message...") 인 경우 트랜잭션을 실패로 만든다.
- require(false, "message")인 경우 트랜잭션을 실패로 만든다.

트랜잭션(Transaction)이란?

트랜잭션이란 사용자가 블록체인에 어떤 요청(명령)을 보내는 행위입니다. 이 요청은 컨트랙트의 함수 실행, 토큰 전송, 컨트랙트 배포 등을 포함합니다.

“블록체인의 상태(state)를 바꾸는 요청” = 트랜잭션

행위	트랜잭션 여부	설명
myContract.vote("홍길동") 호출	O	블록체인 상태 변경 (투표 수 증가)
myContract.balanceOf("0x123") 호출	✗ 아님	상태 변경 없음, 단순 조회 (view)
이더 전송 (send, transfer)	O	잔고 상태 변경 발생
컨트랙트 배포	O	새로운 컨트랙트가 생김 (상태 변경)

트랜잭션(Transaction)이란?

트랜잭션은 실패(failed)로 처리됩니다.

이 말은 **컨트랙트 내부에서 이루어진 모든 변경 사항이 되돌려진다는** 뜻입니다.

(※ 즉, vote 수나 상태변수는 아무것도 안 바뀜)

하지만 실패한 트랜잭션 자체는 블록에 기록됩니다.

```
function vote(string memory name) public {  
    require(!hasVoted[msg.sender], "이미 투표하셨습니다");  
  
    voteCounts[name]++;  
    hasVoted[msg.sender] = true;  
}
```

```
function vote(string memory name) public {  
    voteCounts[name]++;  
    hasVoted[msg.sender] = true;  
  
    require(!hasVoted[msg.sender], "이미 투표하셨습니다");  
}
```

Modifier

modifier는 함수 실행 전에 미리 검사하거나, 전처리 작업을 수행하는 재사용 가능한 코드 블록이다. 조건 검사를 반복적으로 함수마다 쓰지 않고, 공통 로직을 함수 앞에 붙여서 자동 처리할 수 있다.

```
function withdraw() public {  
    require(msg.sender == owner, "너 주인 아니야");  
    // 출금 로직  
}  
  
function changeOwner(address _new) public {  
    require(msg.sender == owner, "너 주인 아니야");  
    // 변경 로직  
}
```

modifier는 Solidity에서 함수 실행 전 조건을 검사하거나, 공통 코드를 붙이는 도구입니다. 함수에 붙여서 재사용성과 보안성을 높이는 데 유용합니다.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract Bank {
    address public owner;
    mapping(address => uint) public balances;

    constructor() {
        owner = msg.sender;
    }

    // 모디파이어 정의
    modifier onlyOwner() {
        require(msg.sender == owner, "너 주인 아니야");
        _; // <- 여기서 함수 본문 실행
    }

    function deposit() public payable {
        balances[msg.sender] += msg.value;
    }

    // 출금 함수는 오직 owner만 실행 가능
    function withdraw(address to, uint amount) public onlyOwner {
        require(balances[to] >= amount, "잔액 부족");
        balances[to] -= amount;
        payable(to).transfer(amount);
    }
}
```


7-1

7-2

Modifier 8-1.8-2

Quiz 6 - 1

```
1  // SPDX-License-Identifier: GPL-3.0
2  pragma solidity >=0.7.0 <0.9.0;
3
4  contract quiz1 {
5
6      /*
7          이벤트명은 Info이며, 매개변수로 uint를 받는 이벤트를 정의하세요.
8          자료형 uint와 가시성 지정장 public을 갖는 변수 num을 정의하세요.
9          변수 num에 값을 대입하는 생성자를 만드세요.
10         변수 num을 변경하는 함수 changeNum을 만드세요.
11         함수 changeNum이 실행될때 변경된 num의 값을 Info이벤트를 통해 블록체인에 출력하세요.
12     */
13 }
```

Quiz 6 - 2

```
1  // SPDX-License-Identifier: GPL-3.0
2  pragma solidity >=0.7.0 <0.9.0;
3
4  contract Math {
5      function add(uint _num1, uint _num2) public pure returns(uint) {
6          return _num1 + _num2;
7      }
8  }
9
10 contract quiz2 {
11
12     /*
13
14     스마트 컨트랙트 Math를 인스턴스화해보세요.
15     Math의 add을 호출하는 quiz2의 함수 addNumbers를 만들어보세요.
16
17     */
18 }
```

Quiz 6 - 3

```
1  // SPDX-License-Identifier: GPL-3.0
2  pragma solidity >=0.7.0 <0.9.0;
3
4  contract Student {
5      function university() public pure virtual returns(string memory) {  infinite gas
6          return "The University of Solidity";
7      }
8  }
9
10 contract quiz3 {
11
12     /*
13      해당 조건에 맞는 스마트 컨트랙트를 만들어 보세요.
14      quiz3 스마트 컨트랙트는 Student 스마트 컨트랙트를 상속합니다.
15      Student 스마트 컨트랙트의 university 함수를 오버라이딩해 The University of Blockchain을 반환하세요.
16     */
17 }
```

Quiz 7 - 1

```
1  // SPDX-License-Identifier: GPL-3.0
2  pragma solidity >=0.7.0 <0.9.0;
3
4  contract quiz1 {
5
6      /*
7          해당 조건에 맞는 스마트 컨트랙트를 만들어 보세요.
8          - 자료형 uint 매개변수 _num을 갖고 있는 함수 revertFunction을 정의하세요.
9          - 매개변수 _num이 6이상일때 revert를 통해 오류를 발생시키세요.
10         - 오류 메시지는 "must not be more than 5" 입니다.
11         - 매개변수 _num이 5 이하라면 입력받은 매개변수를 출력하세요.
12     */
13
14 }
```

Quiz 7 - 2

```
1  // SPDX-License-Identifier: GPL-3.0
2  pragma solidity >=0.7.0 <0.9.0;
3
4  contract quiz2 {
5
6      /*
7          해당 조건에 맞는 스마트 컨트랙트를 만들어 보세요.
8          - 자료형 uint 매개변수 _num을 갖고 있는 함수를 정의하세요.
9          - 매개변수 _num이 6 이상일때 require을 통해 오류를 발생시키세요.
10         - 오류 메시지는 "must not be more than 5" 입니다.
11         - 매개변수 _num이 5 이하라면 입력 받은 매개변수를 출력하세요.
12
13         */
14
15 }
```

Quiz 8 - 1

```
1  // SPDX-License-Identifier: GPL-3.0
2  pragma solidity >=0.7.0 <0.9.0;
3
4  contract quiz1 {
5
6      /*
7          해당 조건에 맞는 모디파이어를 만들어 보세요.
8          - 함수 value의 매개변수 _num이 5를 초과일때 함수가 실행되는 모디파이어를 만들어보세요.
9          - 모디파이어의 이름은 numMoreThan5입니다.
10         - 모디파이어의 오류 메시지는 "_num must be more than 5"입니다.
11      */
12
13      function value(uint _num) public pure numMoreThan5(_num) returns(uint) {
14          return _num;
15      }
16  }
```


Quiz 8 - 2

```
1  // SPDX-License-Identifier: GPL-3.0
2  pragma solidity >=0.7.0 <0.9.0;
3
4      /*
5          해당 조건에 맞는 모디파이어를 만들어 보세요.
6          - 아래 2개의 함수에 공통으로 적용된 require함수를 제거하세요.
7          - 제거한 require를 바탕으로 numMoreThan9이라는 이름의 모디파이어를 생성하세요.
8          - 해당 모디파이어를 아래 2개의 함수에 적용하여 아래 require를 지우기전 코드와 같은
9            기능을 하도록 만들어보세요.
10         */
11
12  contract quiz2 {
13      function minus10(uint _num) public pure returns(uint) {
14          require(_num>=10, "_num must be equivalent to or more than 10");
15          return _num - 10;
16      }
17      function plus10(uint _num) public pure returns(uint) {  infinite gas
18          require(_num>=10, "_num must be equivalent to or more than 10");
19          return _num + 10;
20      }
21  }
```


address 자료형

0x08b04aB8762757AEC347727a8c2AD0DA65D0BA3a

- 20바이트 고정 크기, 16진수로 표기, 40개의 16진수의 조합

종류

- EOA (Externally Owned Account) 외부 소유 계정 전자지갑 주소
- CA (Contract Account) 스마트 컨트랙트 배포시 생성

```
address public myAddr = 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4;
```

(address).balance는 uint의 값으로 현재 주소에 보유한 이더의 잔액을 wei 단위로 표시한다. (1 ether = 10^{18} wei)

payable 키워드

- address 타입 변수와 함수에 적용 가능하다
- 이더를 받기 위해서는 해당 키워드가 적용되어야 한다.

```
address payable public myAddr;
```

- payable 가 함수에 적용되면, 함수 호출과 동시에 이더(msg.value의 값)를 전송한다.
- msg.value가 함수 로직에 있을때에는 payable을 붙여줘야 한다.
- payable 함수가 적용된 함수 호출 버튼은 붉은 색이다.

address 와 address payable

- address: 코드 상에서 Ether 를 보낼 수 없는 일반 전자지갑 주소(식별, 잔액조회 용도)
- address payable: transfer, send, call함수를 이용해서 Ether 전송 메서드 호출 가능 (즉, Ether 송수신이 가능)

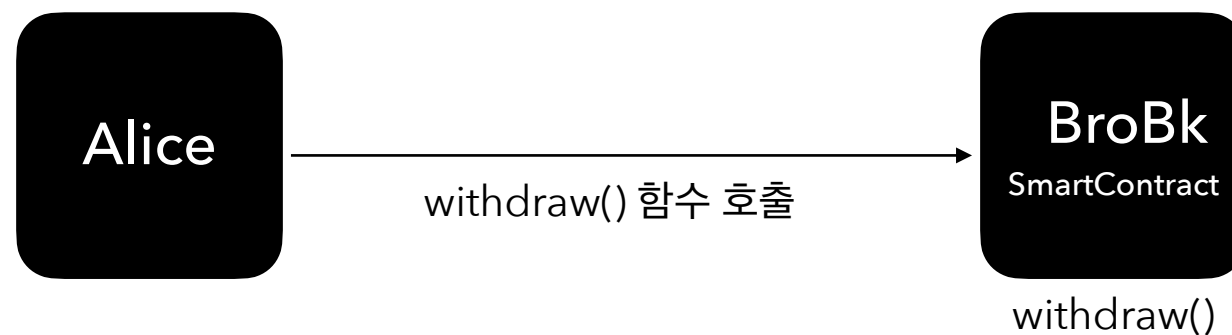
```
address owner;           // 소유자 주소(식별용)
address payable treasury; // 자금을 실제로 받을 지갑
```

```
modifier onlyOwner() {
    require(msg.sender == owner, "Not the owner");
    _;
}
```

```
uint bal = address(this).balance; // 잔액 조회
```

msg.sender msg.value

- 전역 변수!
- msg.sender: 현재 transaction을 발생시킨 address
- msg.value: 함께 전달된 wei의 uint 개수



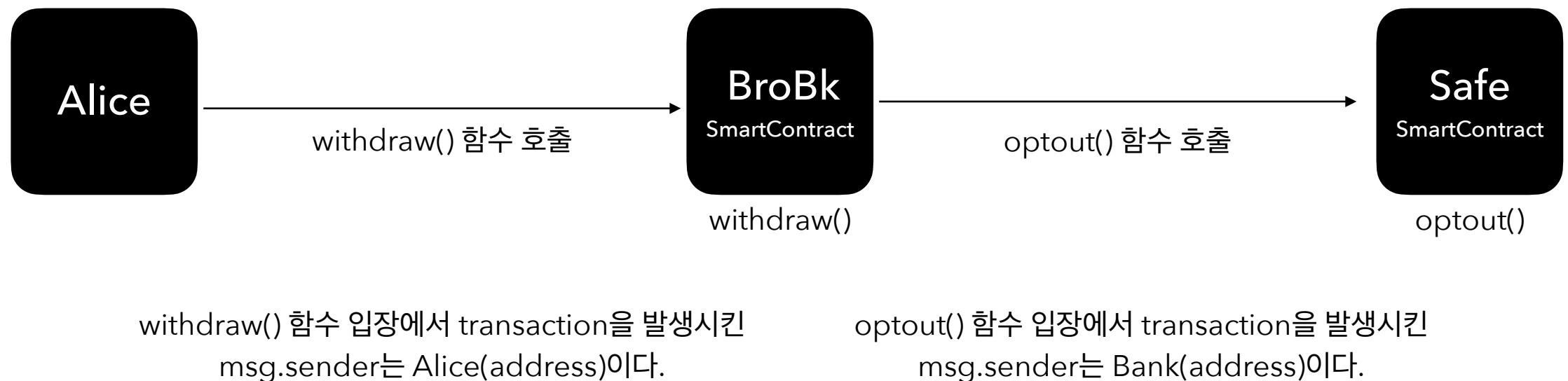
`withdraw()` 함수 입장에서 transaction을 발생시킨 `msg.sender`는 Alice이다.
`msg.sender`의 값은 Alice의 전자지갑 주소이다.

msg.sender

- 스마트컨트랙트를 배포할때 msg.sender의 값은 해당 스마트 컨트랙트를 '배포한' address 이다.
- msg.sender: 현재 transaction을 발생시킨 address

msg.sender

- msg.sender의 값은 상대적인 전자지갑 주소이다



msg.sender

- msg.sender 은 address 타입으로 누가 함수를 호출했는지 알려준다.
- msg.value 는 uint256 타입으로 호출과 함께 얼마나 많은 Ether 를 보냈는지 알려준다. wei 단위로 알려준다.

```
function whoCalledMe() external view returns (address) {  
    return msg.sender;  
}
```

EOA에서 직접 호출하면 msg.sender은 그 지갑의 주소가 넣어진다.
다른 스마트 컨트랙트가 호출하면 msg.sender은 그 스마트 컨트랙트의 주소가 넣어진다.

```
address public owner = msg.sender;  
  
modifier onlyOwner() {  
    require(msg.sender == owner, "Not owner");  
    _;  
}  
  
function changeSetting(uint _v) external onlyOwner {  
    // 오직 배포자(owner)만 수행 가능  
}
```

대표적인 접근 제어

msg.value

- msg.value는 함수가 호출되면서 전송되는 wei 의 양
- 0보다 큰값이면, 함수에 payable이 함수에 적용 필요

msg.value

- msg.value 는 uint256 타입으로 호출과 함께 얼마나 많은 Ether 를 보냈는지 알려준다. wei 단위로 알려준다.
- 함수에 payable 키워드가 없고, msg.value가 0보다 크다면 자동 revert가 발생함! Ether를 받을 함수라면 반드시 payable 키워드가 필수여야함.

최소 금액 입금 예제

```
pragma solidity ^0.8.25;

contract MinDeposit {
    mapping(address => uint) public deposited;
    uint public constant MIN = 0.01 ether;    // 0.01 ETH

    // 돈을 받는 함수이므로 payable
    function deposit() external payable {
        require(msg.value >= MIN, "Need >= 0.01 ETH");
        deposited[msg.sender] += msg.value;
    }
}
```

개별 입출금 예제

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.25;

contract SavingsBox {
    address public owner;
    mapping(address => uint) public balanceOf;

    constructor() {
        owner = msg.sender;
    }

    // (1) 입금 함수 - 누구나
    function deposit() external payable {
        require(msg.value > 0, "Zero deposit");
        balanceOf[msg.sender] += msg.value;    // msg.value 누적 저장
    }

    // (2) 출금 함수 - 내 잔액만 인출
    function withdraw(uint amount) external {
        require(balanceOf[msg.sender] >= amount, "Insufficient");
        balanceOf[msg.sender] -= amount;

        // 실제 송금 (msg.value 사용 안 함)
        (bool ok, ) = payable(msg.sender).call{value: amount}("");
        require(ok, "Transfer failed");
    }

    // (3) 계약 전체 잔액 조회
    function contractBalance() external view returns (uint) {
        return address(this).balance;    // msg.value 아님, 현재 보유액
    }
}
```

msg.value

함수에 payable만 적어 두면 자동으로 돈이 받은 사람에게 넘어간다?

아니다. payable은 “받을 수 있음을 허용”만 한다. 받은 돈을 저장하거나 즉시 송금하는 로직은 개발자가 직접 작성해야한다.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.25;

/*
 * “payable이면 알아서 주인 지갑으로 가겠지?” 하고
 * 아무 코드도 안 넣은 전형적인 실수 예제.
 */
contract LockedTipBox {
    address public owner; // 주인의 지갑 주소

    constructor() {
        owner = msg.sender;
    }

    // (1) 누구나 팁을 보낼 수 있도록 payable
    function tip() external payable {
        // ❌ 받은 Ether를 저장·송금하는 코드가 전혀 없음
        // => Ether는 contract 주소에 쌓이기만 하고 빠져나가지 못함
    }

    // (2) 컨트랙트 내부 잔액 확인용
    function contractBalance() external view returns (uint) {
        return address(this).balance;
    }
}
```

msg.value

함수에 payable만 적어 두면 자동으로 돈이 받은 사람에게 넘어간다?

아니다. payable은 “받을 수 있음을 허용”만 한다. 받은 돈을 저장하거나 즉시 송금하는 로직은 개발자가 직접 작성해야한다.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.25;

contract ForwardingTipBox {
    address payable public owner;

    constructor() {
        owner = payable(msg.sender);
    }

    /*
     * tip()을 호출하면 넘어온 Ether(msg.value)를
     * 즉시 owner 지갑으로 전달한다.
     */
    function tip() external payable {
        require(msg.value > 0, "No tip");

        // 안전한 저수준 호출로 송금
        (bool ok, ) = owner.call{value: msg.value}("");
        require(ok, "Transfer failed");
    }
}
```

9-1

```
1  // SPDX-License-Identifier: GPL-3.0
2  pragma solidity >=0.7.0 <0.9.0;
3
4  contract Ex9_1 {
5
6      function getBalance(address _address) public view returns(uint) {
7          return _address.balance;
8      }
9
10     // 이 함수를 실행하면서 이더를 보낸다. 그 전송한 금액을 표시하는 함수이다.
11     function getMsgValue() public payable returns(uint) {
12         return msg.value;    290 gas
13     }
14 }
```

VALUE

Gwei

CONTRACT

EX9_1 AT 0X1FB...84A9F (BLOCK)

Balance: 0.000010002 ETH

getMsgValue

getBalance 0x1FbB3976F9070a97e49FEe

0: uint256: 10002000000000

Low level interactions

CALLDATA

Transact

스마트 컨트랙트 배포자

스마트 컨트랙트 배포자만 실행 할 수 있는 함수를 만들 수 있을까?

Q.9-1

아래 주어진 코드에서 해당 스마트 컨트랙트를 배포한 사람만 getBalance 함수를 정상적으로 실행할 수 있고, 그외의 사람은 오류 메시지가 표시되도록 하려면?
(tip: constructor와 require를 이용하면 되요)

```
1  // SPDX-License-Identifier: GPL-3.0
2  pragma solidity >=0.7.0 <0.9.0;
3
4  contract Ex9_1 {
5      event SenderAddress(address indexed sender, uint amount);
6
7
8      function getBalance(address _address) public view returns(uint) {
9          return _address.balance;
10     }
11
12     // 이 함수를 실행하면서 이더를 보낸다. 그 전송한 금액을 표시하는 함수이다.
13     function getMsgValue() public payable returns(uint) {
14         return msg.value;    290 gas
15     }
16
17     function getMsgValue2() public payable returns(address) {
18         emit SenderAddress(msg.sender, msg.value); // 이벤트 발생시키기
19         return msg.sender;
20     }
21 }
```


send /transfer / call

이더를 보내는 함수이다.
주소형 타입(Address)의 내장 함수이다.

send / call

- send, call 함수는 항상 require 함수와 같이 사용해야 한다!
- gas의 양에 따른 fallback, receive 함수가 구현되어야 한다.
- send 함수는 gas양이 2300gwei로 제한되므로 더 복잡한 로직을 수행하면 실패가 발생한다.
- call 함수는 gas양을 가변적으로 정할 수 있어 send 함수보다 더 복잡한 로직을 수행 할 수 있다.
- send 함수로 이더를 받으려면 payable이 필수이다.
- call 함수로 이더를 받는다면 꼭 payable 필수가 아니다.
- 최근에는 call 함수 사용 권장함

transfer

- transfer는 require로 점검이 필요없다.
- gas의 양에 따른 fallback, receive함수가 구현되어야 한다.
- transfer 함수는 gas양이 2300gwei로 제한되므로 더 복잡한 로직을 수행하면 실패가 발생한다.
- transfer로 받는 주소는 payable가 필수이다.

send / call / transfer

표 9.4 send/call/transfer 특징

함수	보내는 주소의 payable 여부	가스 소비량	실패 시
send	0	2300 gwei	false 반환으로 인해 require 필요
call	X	가변적임	false 반환으로 인해 require 필요
transfer	0	2300 gwei	트랜잭션 실패

send / call / transfer

```
payable ( address ).send(1000);
```

단위는 wei

```
payable ( address ).transfer(1000);
```

단위는 wei

```
address.call{value:1000}("");
```

단위는 wei

용돈 관리 (transfer / send / call)

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.25;

contract KidWallet {
    address public kid;          // 아이 주소 (수령인)
    address public parent;       // 부모 주소 (관리자)

    constructor(address _kid) {
        kid = _kid;
        parent = msg.sender;     // 배포자가 부모
    }

    // -----
    // 1) 부모가 용돈 입금
    // -----
    function give() external payable {
        require(msg.sender == parent, "Only parent");
        require(msg.value > 0, "Need some ETH");
        // 돈은 일단 계약에 쌓임
    }

    // -----
    // 2-A) transfer 로 주기
    // -----
    function withdrawByTransfer() external {
        require(msg.sender == kid, "Only kid");
        uint amount = address(this).balance;

        // ① transfer (가스 2300, 실패하면 자동 revert)
        payable(kid).transfer(amount);
    }
```

```
// -----
// 2-B) send 로 주기
// -----
function withdrawBySend() external {
    require(msg.sender == kid, "Only kid");
    uint amount = address(this).balance;

    // ② send (가스 2300, bool 반환)
    bool ok = payable(kid).send(amount);
    require(ok, "Send failed");
}

// -----
// 2-C) call 로 주기
// -----
function withdrawByCall() external {
    require(msg.sender == kid, "Only kid");
    uint amount = address(this).balance;

    // ③ call (가스 전달, bool+data 반환)
    (bool ok, ) = payable(kid).call{value:
amount}("");
    require(ok, "Call failed");
}
}
```

```

1  // SPDX-License-Identifier: GPL-3.0
2  pragma solidity >=0.7.0 <0.9.0;
3
4  contract Ex9_3 {
5
6      function getBalance(address _address) public view returns(uint) {
7          return _address.balance;
8      }
9
10     function etherUnits() public pure returns(uint,uint,uint) {    ⚙ inf.
11         return(1 ether, 1 gwei, 1 wei);
12     }
13
14     function ethDelivery0() public payable returns(uint) {    ⚙ 335 gas
15         return msg.value;
16     }
17
18     function ethDelivery1(address payable _address) public payable {
19         bool result = _address.send(123 gwei);
20         require(result, "Failed");
21     }
22
23     // 0.01 = 10^7gwei 은 보내야 확인 가능
24     function ethDelivery2(address _address) public payable {    ⚙ infinit
25         payable(_address).transfer(msg.value);
26     }
27

```

msg.value의 값을

- 설정된 금액보다 큰 금액으로 설정한 경우
- 설정된 금액보다 작은 금액을으로 설정한 경우

Q9-2. ethDelivery1 함수를 호출할 때 msg.value가 고정 전송액인 123 gwei를 초과하면, 초과된 금액을 즉시 msg.sender에게 자동으로 환불하도록 코드로 수정하십시오.

ethDelivery1 함수를 수정하여

1. "호출자가 보내는 금액(msg.value)이 최소 123 gwei인지 먼저 확인하고,"
2. "정확히 123 gwei를 수신자 주소(_address)에 송금한 뒤,"
3. "남는 금액이 있으면 즉시 msg.sender (호출자)에게 환불하도록 구현하십시오."

fallback, receive

이더리움 스마트 컨트랙트에서 이더를 받을 수 있는 방법에는 두 가지가 있습니다. 하나는 payable 함수를 통한 방법이고, 다른 하나는 fallback 함수나 receive 함수를 통한 방법입니다.

1. payable 함수: 이 함수는 이더리움을 명시적으로 받을 수 있도록 설정된 함수입니다. 호출자가 이더리움을 보내면서 이 함수를 직접 호출할 수 있습니다. 예를 들어, 위에서 언급한 ethDelivery1이나 ethDelivery2와 같은 payable 함수가 이에 해당합니다. 이 함수들은 트랜잭션의 일부로 이더를 전송받고, 그 이더를 처리하는 로직을 포함할 수 있습니다.
2. fallback, receive 함수
fallback 함수: 이 함수는 이름이 지정되지 않은 함수로, 스마트 컨트랙트로 이더를받거나, 존재하지 않는 함수를 호출했을 때 실행됩니다. 이 함수는 payable로 선언되어야 이더를 받을 수 있습니다.
receive 함수: Solidity 0.6.0 이후에 도입된 이 함수는 오직 순수 이더(콜 데이터 없이)를 받을 때만 호출됩니다. 이 함수 역시 payable로 선언되어야 합니다.

payable 함수를 직접 호출할 수 있었다면, 그 함수가 이더를 수락할 수 있도록 설정되어 있기 때문에 fallback 함수나 receive 함수 없이도 이더를 받을 수 있었던 것입니다. 반면, fallback 또는 receive 함수는 스마트 컨트랙트가 일반적인 이더 전송(함수를 호출하지 않고 단순히 주소로 이더를 보낼 경우)을 받기 위해 필요합니다. 이 함수들이 없다면, 스마트 컨트랙트는 그런 방식의 이더 전송을 거부하게 됩니다.

fallback, receive

- receive는 스마트 컨트랙트가 이더를 받을 수 있도록 한다.
- fallback은 스마트 컨트랙트에 존재하지 않는 함수가 호출되었을때 수행된다.
- fallback에 payable을 적용하면 이더 수신도 가능하다.

fallback

- 스마트 컨트랙트가 이더를 받을때 fallback 함수가 수행됨

(send, transfer을 통해 이더를 받는 경우 fallback함수가 2300 gwei를 소비해 자신의 로직을 수행하는 것이다.)

- 무기명 함수, external로 지정, 매개변수가 없음

- 반환값이 없음

- payable이 적용되면 이더 수신 가능함

- send, transfer 수행시 2300 gwei 만큼만 수행된다.

```
function() external payable {  
    :  
    :  
}
```

0.6이하

```
fallback() external payable {  
    :  
    :  
}
```

0.7이상

receive

- 무기명 함수, external로 지정, 매개변수가 없음
- 반환값이 없음, payable 필수,
- function 키워드는 안 붙인다.
- 오버라이딩, 모디파이어 적용 가능

```
receive() external payable {  
    :  
    :  
}
```

스마트 컨트랙트가 이더를 받을 수 있게 하기위해서는 receive 함수를 정의해야한다. send 또는 transfer함수를 통해 이더를 받은 경우 2300gwei의 가스를 소비해 receive함수가 실행된다.

fallback (0.6미만)

- 이름이 존재하지 않는 무기명 함수
- external를 필수로 붙여야함
- 매개변수가 없다. 반환값이 없다.
- payable이 적용되어야 이더를 받을 수 있다.
- 스마트컨트랙트가 이더를 받으면 payable이 적용된 fallback 이 실행된다. send와 transfer을 통해 이더를 받는 경우 fallback 함수는 2300gwei만큼의 가스를 소비해 함수의 코드를 수행한다. 가스비가 충분하지 않은 경우 call함수를 호출해서 이더를 보내야 한다.
- fallback함수가 payable가 제공되지 않는다면 이더를 받을 수 없다.
- 하나의 스마트 컨트랙트는 하나의 fallback만 가능하다.

```
1 // SPDX-License-Identifier: GPL-3.0
2 pragma solidity >=0.4.0 <0.6.0;
3
4 contract Ex9_5 {
5
6     event Obtain(address from, uint amount);
7     function() external payable {  undefined gas
8         emit Obtain(msg.sender, msg.value);
9     }
10
11     function getBalance(address _address) public view returns(uint) {
12         return _address.balance;
13     }
14
15     function sendEther(address payable _address) public payable {  undefined gas
16         _address.transfer(msg.value);
17     }
18
19 }
```

- 어떻게 구분할 것인가?

```
1  pragma solidity ^0.5.0;
2
3  contract Example {
4      // 이더 수신 이벤트
5      event Received(address sender, uint256 amount);
6
7      // 존재하지 않는 함수 호출 이벤트
8      event FallbackCalled(address sender, bytes data, uint256 amount);
9
10     // 이더를 수신하거나 존재하지 않는 함수가 호출될 때 실행되는 fallback 함수
11     function() external payable {
12         if (msg.data.length == 0) {
13             // msg.data가 비어 있으면 이더를 수신한 것으로 간주 ⚠ undefined gas
14             emit Received(msg.sender, msg.value);
15         } else {
16             // msg.data가 비어 있지 않으면 존재하지 않는 함수가 호출된 것으로 간주
17             emit FallbackCalled(msg.sender, msg.data, msg.value);
18         }
19     }
20 }
```

receive (0.6 이상)

- 함수를 시작하는 키워드 `function`이 없이 `receive()`만 입력한다.
 - `receive`에는 `external`을 필수로 붙여야 한다.
 - `payable`를 필수로 붙여야 한다.
 - 매개변수가 존재하지 않는다.
 - 반환값이 존재하지 않는다.
 - 하나의 `receive`함수만 존재할 수 있다.
 - 스마트 컨트랙트가 이더를 받으면 `receive`함수가 실행된다.
- `send`와 `transfer` 함수를 통해 이더를 받는 경우 2300 gwei만큼의 가스를 소비해 `receive` 함수가 실행된다.

129

receive 테스트

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.25;

contract SimpleReceiver {
    event Deposit(address indexed from, uint256 indexed amount);

    /* calldata 0바이트 + ETH 전송 → 여기로 진입 */
    receive() external payable {
        emit Deposit(msg.sender, msg.value);
    }

    /* 잔액 조회용 */
    function contractBalance() external view returns (uint256) {
        return address(this).balance;
    }
}
```


receive 테스트

0x0d12767221A1AdbF5a60e3A075efC49cF1f9c7F3

Deployed Contracts 1

▼ SIMPLERECEIVER AT 0X0D1.

contractBalan...

0: uint256: 0

Low level interactions

CALLDATA

Transact

version=soljson-v0.8.30+commit.73712... ☆ ⓘ

< 보내기

발신

Account 1
0xb3af8...D824d

S Sepoli... 0.001

잔액: 0.14039616 최대

수신

0x0d127...9c7F3
0x0d127...9c7F3

S Sepoli... 0.001

취소

계속

receive 테스트

0x0d12767221A1AdbF5a60e3A075efC49cF1f9c7F3

Contract

0x0d12767221A1AdbF5a60e3A075efC49cF1f9c7F3

Overview

ETH BALANCE
0.001 ETH

More Info

CONTRACT CREATOR
0xb3af807a...1711D824d | 4 mins ago

Multichain Info

N/A

Transactions

Token Transfers (ERC-20)

Contract

Events

Latest 1 from a total of 1 transactions

Download Page Data

Transaction Hash	Method	Block	Age	From	To	Amount	Txn Fee
0x97a874416b...	Transfer	8365727	2 mins ago	0xb3af807a...1711D824d	IN 0x0d127672...cF1f9c7F3	0.001 ETH	0.0000339

[Download: CSV Export]



Transaction Details

Overview Logs (1) State

[This is a Sepolia Testnet transaction only]

Transaction Hash:	0x97a874416b0b8ed82bb8126dc2bead8bb654373c4b7299890f7a103ce8e9b389
Status:	Success
Block:	8365727 3 Block Confirmations
Timestamp:	35 secs ago (May-20-2025 05:37:00 AM UTC)
From:	0xb3af807aDa0E13af3b596D01deaD6b71711D824d
To:	0x0d12767221A1AdbF5a60e3A075efC49cF1f9c7F3
Value:	0.001 ETH
Transaction Fee:	0.000033909392500352 ETH
Gas Price:	1.500946906 Gwei (0.000000001500946906 ETH)

More Details: [Click to show more](#)



Transaction Details

Overview Logs (1) State

Transaction Receipt Event Logs

89	Address	0x0d12767221a1adbf5a60e3a075efc49cf1f9c7f3
	Name	Deposit (address, uint256) View Source
	Topics	0 0xe1fffcc4923d04b559f4d29a8bfc6cda04eb5b0d3c460751c2402c5c5cc9109c 1 → 0x000000000000000000000000b3af807ada0e13af3b596d01dead6b71711d824d 2 → 0x0038d7ea4c68000
	Data	0x

fallback (0.6 이상)

- 함수를 시작하는 키워드 `function`이 없이 `fallback()`만 입력한다.
- `fallback` 함수에 가시성 지정자 `external`은 필수이다.
- `payable`은 선택적으로 붙일 수 있다. `fallback` 함수에 `payable`을 적용하고, `receive` 함수가 정의되지 않았다면 `fallback` 함수를 통해 이더를 받을 수 있다. `send`와 `transfer`을 통해 이더를 받는 경우 `fallback` 함수는 2300gwei가스를 소비해 실행된다.
- `fallback` 함수는 매개변수와 반환값이 존재하지 않는다.
- 하나의 `fallback` 함수만 존재할 수 있다.

- 0.60이상에서 payable이 적용된 fallback함수로 이더 받기

```
1 // SPDX-License-Identifier: GPL-3.0
2 pragma solidity >=0.7.0 <0.9.0;
3
4 contract Ex9_7 {
5
6     event Obtain(address from, uint amount);
7     fallback() external payable { undefined gas
8         emit Obtain(msg.sender, msg.value);
9     }
10
11     function getBalance() public view returns(uint) { 339 gas
12         return address(this).balance;
13     }
14
15     function sendEther() public payable { infinite gas
16         payable(address(this)).transfer(msg.value);
17     }
18
19 }
```

생성자에 payable이 적용되면 스마트 컨트랙트 배포와 동시에 그 스마트 컨트랙트로 이더를 보낼 수 있다.

```
1 // SPDX-License-Identifier: GPL-3.0
2 pragma solidity >=0.7.0 <0.9.0;
3
4 contract Ex9_8 {
5
6     constructor() payable {  36263 gas 36200 gas
7
8     }
9
10    function getBalance() public view returns(uint) {
11        return address(this).balance;
12    }
13 }
```

만약 여기에 receive함수를 정의한다면, 실행이 될까?

만약 여기에 receive함수를 정의한다면, 실행이 될까?

```
1 // SPDX-License-Identifier: GPL-3.0
2 pragma solidity >=0.7.0 <0.9.0;
3
4 contract Ex9_8 {
5
6     event ReceiveEvent(address adr);
7
8     constructor() payable { ⛛ 70093 gas 70000 gas
9
10    }
11
12    receive() external payable { ⛛ undefined gas
13        emit ReceiveEvent(msg.sender);
14    }
15
16
17    function getBalance() public view returns(uint) {
18        return address(this).balance;
19    }
20 }
21
```

아래 계약을 두번 배포한다. 그리고 오른쪽 그림과 같이 1 gwei를 보낸다.

```
1  // SPDX-License-Identifier: GPL-3.0
2  pragma solidity >=0.7.0 <0.9.0;
3
4  contract Ex9_6 {
5
6      event Obtain(address from, uint amount);
7      receive() external payable {  undefined gas
8          emit Obtain(msg.sender, msg.value);
9      }
10
11     function getBalance() public view returns(uint) {  317 gas
12         return address(this).balance;
13     }
14
15     function sendEther(address toaddr) public payable {  infinite ga
16         payable(toaddr).transfer(msg.value);
17     }
18
19 }
```

```
decoded input
{
  "address toaddr": "0x3186afA84B143C880CCb28A7D4ca713E13765937"
}
```

```
logs
[
  {
    "from": "0x3186afa84b143c880ccb28a7d4ca713e13765937",
    "topic": "0x0368838220383a15f8cb55f018fd7c72a2a7b39241ce69b3633d92b2dcb3aca7",
    "event": "Obtain",
    "args": {
      "0": "0x53A948391D646653f7bE3d31C299916033D16984",
      "1": "1000000000",
      "from": "0x53A948391D646653f7bE3d31C299916033D16984",
      "amount": "1000000000"
    }
  }
]
```

VALUE

1

Gwei

CONTRACT

EX9_6 AT 0X53A...16984 (BLOCKCHAIN)

Balance: 0 ETH

sendEther

0x3186afA84B143C880CCb28A7D4ca7

getBalance

0: uint256: 0

Low level interactions

CALLDATA

Transact

EX9_6 AT 0X318...65937 (BLOCKCHAIN)

Balance: 0.000000001 ETH

sendEther

address toaddr

getBalance

0: uint256: 0

call

1. 이더를 보낼 때

```
송금받는주소.call{value:1000}("");
```

```
스마트컨트랙트주소.call{value:1000}("");
```

(0.7이상)

```
스마트컨트랙트주소.call.value(1000)("");
```

(0.7미만)

2. 외부 스마트 컨트랙트 함수를 호출할 때

```
스마트컨트랙트주소.call(abi.encodeWithSignature("add(uint256,uint256)", 10, 15));
```

이부분은 특히 띄어쓰기 없이 명시해야함

uint 축약은 안되고 반드시 명시적으로 uint256이라고 써야함


```

1  // SPDX-License-Identifier: GPL-3.0
2  pragma solidity >=0.7.0 <0.9.0;
3
4  contract AddContract {
5
6      uint result = 0;
7
8      function add(uint256 _num1, uint256 _num2) public {    infinite gas
9          result = _num1 + _num2;
10     }
11
12     function returnResult() public view returns(uint) {    2415 gas
13         return result;
14     }
15 }
16
17
18 contract CallerContract {
19
20     function addButtons(address _address, uint _num1, uint _num2) public {    infinite gas
21         (bool success, ) = _address.call(abi.encodeWithSignature("add(uint256,uint256)",_num1 ,_num2));
22         require(success, "Failed");
23     }
24
25     function showResult(address _address) public returns(bytes memory) {    infinite gas
26         (bool success, bytes memory result) = _address.call(abi.encodeWithSignature("returnResult()"));
27         require(success, "Failed");
28         return result;
29     }
30 }
31

```

1. CallerContract의 addButtons을 호출할때 "AddContract" Deploy 주소를 넣고 매개변수를 넣어 호출한다.
2. CallerContract의 showResult를 호출할때 "AddContract" Deploy 주소를 넣고 호출한다.

call

- * 이더를 받는 address 타입의 주소는 payable 키워드를 적용할 필요가 없다.
 - * 이더의 송금 결과를 true, false로 반환한다. false인 경우 require로 트랜잭션을 반드시 실패로 만들어야 한다.
 - * 함수 fallback 또는 receive의 가스 소비량이 가변적이다. 함수 send를 통해 이더를 스카트 컨트랙트로 보내면 이 스마트 컨트랙트에 정의된 fallback 또는 receive 함수가 최대 2300 gwei를 의존해 함수의 로직을 실행합니다. 그러나 2300 gwei를 통해 할 수 있는 일은 매우 제한적으로 call 함수가 있는 것이다.
call 함수는 send 함수와 다르게 가스의 제한량이 따로 없어 소비할 수 있는 모든 가스를 소비할 수 있다. 즉 가스의 소비량이 fallback 또는 receive 함수의 로직에 따라 가변적으로 변한다. 이러한 특징으로 send 함수를 쓸때보다 fallback 또는 receive 함수의 조직으로 더 많은 것을 할 수 있다.
- send, transfer은 fallback 또는 receive함수의 가스 소비량을 2300 gwei로 제한한다. send 함수로 특정 스마트 컨트랙트로 이더를 보낸다면, 함수 fallback, receive함수는 최대 2300gwei 가스를 소비해 자신의 로직을 실행시킬 수 있다.

```
1 // SPDX-License-Identifier: GPL-3.0
2 pragma solidity >=0.7.0 <0.9.0;
3
4 contract Ex9_2 {
5
6     address public owner;
7     modifier onlyOwner() {
8         require(owner == msg.sender, "Error: caller is not the owner");
9         _;
10    }
11    constructor () { 218304 gas 193800 gas
12        owner = msg.sender;
13    }
14
15    function getBalance(address _address) public view onlyOwner() returns(uint) { 5446 gas
16        return _address.balance;
17    }
18
19    function getMsgValue() public payable onlyOwner() returns(uint) { 2579 gas
20        return msg.value;
21    }
22 }
```

```
1 // SPDX-License-Identifier: GPL-3.0
2 pragma solidity >=0.7.0 <0.9.0;
3
4 contract Ex9_3 {
5
6     function getBalance(address _address) public view returns(uint) {    ⛽ 3279 gas
7         return _address.balance;
8     }
9
10    function etherUnits() public pure returns(uint,uint,uint) {    ⛽ infinite gas
11        return(1 ether, 1 gwei, 1 wei);
12    }
13
14    function ethDelivery1(address payable _address) public payable {    ⛽ undefined gas
15        bool result = _address.send(10 ether);
16        require(result, "Failed");
17    }
18
19    function ethDelivery2(address _address) public payable {    ⛽ infinite gas
20        payable(_address).transfer(msg.value);
21    }
22
23 }
```

해당 _address는 따로 payable을 명시하지 않아도 된다. call 함수의 value는 송금할 wei를 나타내며, call의 value는 # msg.value이므로 ethDelivery1함수가 실행될때 전송된 wei가 _address로 보내질 것이다. call 함수의 가스 양은 gas가# 나타내며 가스를 제공받는 대상은 fallback 또는 receive이다. 3000gwei를 fallback 또는 receive함수에 3000gwei를 # 제공한다. 따로 명시하지 않는다면 사용할 수 있는 모든 가스를 fallback 또는 receive함수에 제공한다.

```
1 // SPDX-License-Identifier: GPL-3.0
2 pragma solidity >=0.5.0 <0.9.0;
3
4 contract Ex9_4 {
5
6     function getBalance(address _address) public view returns(uint) { 3257 gas
7         return _address.balance;
8     }
9
10    function ethDelivery1(address _address) public payable { infinite gas
11        (bool result, ) = _address.call{value:msg.value, gas: 30000}("");
12        require(result, "Failed");
13    }
14    /*
15    function ethDelivery2(address _address) public payable {
16        (bool result, ) = _address.call.value(msg.value).gas(30000)("");
17        require(result, "Failed");
18    }
19    */
20 }
```

call 함수는 2개의 반환값을 갖는다. 첫번째는 call 함수의 성공여부를 true, false의 값이고, 두번째는 호출한 함수의 결과값을 bytes 형으로 반환한다. 여기서는 함수를 호출하지 않았으므로 두번째 값은 받지 않았다.

Quiz 9 - 3

해당 스마트 컨트랙트의 잔액을 리턴하는 함수를 만드세요.

`address(this)` : 이렇게 하면 스마트 컨트랙트 주소 반환함!

Quiz 9 - 4

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity >=0.7.0 <0.9.0;

contract Ex9_6 {

    event Obtain(address from, uint amount);
    receive() external payable {  undefined gas
        emit Obtain(msg.sender, msg.value);
    }

    function getBalance() public view returns(uint) {  317 gas
        return address(this).balance;
    }

    function sendEther(address toaddr) public payable {  infinite gas
        payable(toaddr).transfer(msg.value);
    }

    function sendEther2() public payable {  infinite gas
        payable(address(this)).transfer(msg.value);
    }
}
```

Remix IDE에서 sendEther2()함수 버튼을 클릭한 경우 receive() 함수의 Obtain 이벤트에 msg.sender는 어떠한 내용이 출력되는가? 또한 왜 그 내용이 출력되는지를 코드에 주석을 달아 스크린샷으로 제출하세요.

Quiz 9 - 5

```
1  // SPDX-License-Identifier: GPL-3.0
2  pragma solidity >=0.7.0 <0.9.0;
3
4  contract quiz1 {
5      function sendEther(address _address) public payable {  409 gas
6
7      /*
8      함수의 로직을 완성시키세요.
9      - 함수 sendEther는 msg.value만큼 이더를 전송하는 함수이다.
10     - 함수 call을 이용해 sendEther를 완성하세요.
11     */
12
13     }
14 }
15
```


Quiz 9 - 6

```
1  // SPDX-License-Identifier: GPL-3.0
2  pragma solidity >=0.7.0 <0.9.0;
3
4  /*
5   - quiz2 스마트 컨트랙트에 receive 함수를 정의하세요.
6   - 함수 getBalance가 quiz2의 잔액을 반환하게 함수를 완성하세요.
7   - 함수 sendEther는 quiz2에게 이더를 보내는 함수이며, transfer 함수를 이용해서 완성해보세요.
8  */
9
10 contract quiz2 {
11     ✓
12     function getBalance() public view returns(uint) {
13         /*
14         함수의 로직을 완성시키세요.
15         */
16     }
17     function sendEther() public payable { 329 gas
18         /*
19         함수의 로직을 완성시키세요.
20         */
21     }
22 }
23
```

Quiz 9 - 7

```
1  // SPDX-License-Identifier: GPL-3.0
2  pragma solidity >=0.7.0 <0.9.0;
3
4  contract Math {   uint256      uint256
5      function mul(uint _num1, uint _num2) public pure returns(uint) {
6          return _num1 * _num2;
7      }
8  }
9
10 contract quiz3 {
11
12     uint256      uint256
13     function CallMath(address _address, uint _num1, uint _num2) public returns(bytes memory) {
14
15         /*
16         함수의 로직을 완성시키세요.
17         - 스마트컨트랙트 quiz3 함수 CallMath는 Math 스마트 컨트랙트의 mul함수를 호출한다.
18         - 함수 call을 이용해 함수 CallMath의 로직을 완성하세요.
19         */
20
21         require(success, "Failed");
22         return output;
23     }
24 }
```

```
function showResult(address _address) public returns(bytes memory) {
    (bool success, bytes memory result) = _address.call(
        abi.encodeWithSignature("returnResult()")
    );
    require(success, "Failed");
    return result;
}
```

테스트 네트워크에 적용

DEPLOY & RUN TRANSACTIONS

0 Wei

CONTRACT (Compiled by Remix)

Pre1 - Pre1.sol

Deploy

☐ Publish to IPFS

OR

At Address Load contract from Address

Transactions recorded 1

Deployed Contracts

PRE1 AT 0X709...57611 (BLOCKCHAIN)

Balance: 0.002 ETH

getbalance

0: uint256: 2000000000000000

owner

0: address: 0x4ffF727b33b76E92257aF64A58fCa56F0FEE67AD

Home Pre1.sol

```
1 // SPDX-License-Identifier: GPL-3.0
2
3 pragma solidity >=0.7.0 < 0.9.0;
4
5 contract Pre1 {
6     address public owner;
7
8     event paidAddress(address sender, uint256 payment);
9
10    modifier onlyOwner() {
11        require(msg.sender == owner, "Ownable: caller is not the owner");
12    }
13
14    constructor(){
15        owner = msg.sender;
16    }
17
18    receive() external payable {
19        require(msg.value == 10**15, "Must be 0.001 ether.");
20        emit paidAddress(msg.sender, msg.value);
21    }
22
23    function getbalance() public view onlyOwner() returns(uint256){
24        return address(this).balance;
25    }
26
27 }
28
29
```

Sepolia 테스트 네트워크

Account 3
0x4ff...67AD

0.0965 SepoliaETH

구매 보내기 스왑 Bridge

자산 NFT 활동

계약 상호 작용

May 31 · metamask

-0.001 Sepol...
-0.001 SepoliaETH

계약 상호 작용

상태 확인됨

발신 수신

거래

임시값 30

금액 -0.001 SepoliaETH

가스 한도 (단위) 34011

가스 사용됨 (단위) 22674

기본 수수료(GWEI) 0.000000008

우선 수수료(GWEI) 1.5

총 가스 수수료 0.000034 SepoliaETH

가스당 최대 수수료 0.000000002 SepoliaETH

합계 0.00103401 SepoliaETH

DEPLOY & RUN TRANSACTIONS

ENVIRONMENT

Injected Provider - MetaMask

Injected Provider - MetaMask

Remix VM (Shanghai)

Remix VM (Merge)

Remix VM (London)

Remix VM (Berlin)

테스트 네트워크에 적용

Sepolia Testnet

Search by Address / Txn ID

Etherscan

Transaction Details < >

Overview Logs (1) State

[This is a Sepolia Testnet transaction only]

Transaction Hash:

0xc99627dcc7e91e006b78f0edb2c6ff0348bea32879f2499ab1a9cd0f6648ecc4

Status:

Success

Block:

3592169 32 Block Confirmations

Timestamp:

6 mins ago (May-30-2023 11:15:48 PM +UTC)

From:

0x4fff727b33b76E92257aF64A58fCa56F0FEE67AD

To:

0x70979003e614b50FD71FEe179CC2576Da5E57611

Value:

0.001 ETH (\$0.00)

Transaction Fee:

0.000034011000181392 ETH (\$0.00)

Gas Price:

1.500000008 Gwei (0.000000001500000008 ETH)

More Details: + Click to show more

Sepolia 테스트 네트워크

Account 3
0x4ff...67AD

연결되지 않음

0.0965 SepoliaETH

+ 구매

↗ 보내기

↔ 스왑

↻ Bridge

자산

NFT

활동

계약 상호 작용

May 31 · metamask

-0.001 Sepol...

-0.001 SepoliaETH

계약 상호 작용

실패 · metamask

-0 SepoliaE...

-0 SepoliaETH

테스트 네트워크에 적용

Sepolia Testnet

Search by Address / Txn Hash / Block / Token

Etherscan

HomeBlockchainTokensNFTsMisc

Contract

0x70979003e614b50FD71FEe179CC2576Da5E57611

Overview

ETH BALANCE
0.002 ETH

More Info

CONTRACT CREATOR
0x4ffF72...0FEE67AD at txn 0x54984677b0cb3850...

Multi Chain

MULTICHAIN ADDRESSES
1 address found via Blockscan

Transactions

Token Transfers (ERC-20)

Contract

Events

Latest 4 from a total of 4 transactions

Transaction Hash	Method	Block	Age	From	To	Value	Txn Fee
0xc99627dcc7e91e00...	Transfer	3592169	7 mins ago	0x4ffF72...0FEE67AD	IN 0x709790...a5E57611	0.001 ETH	0.00003401
0xc7f4333326d8923f4...	Transfer	3592166	8 mins ago	0x4ffF72...0FEE67AD	IN 0x709790...a5E57611	0 ETH	0.0000315
0x91c21e3eddb71a8c...	Transfer	3592157	10 mins ago	0x4ffF72...0FEE67AD	IN 0x709790...a5E57611	0.001 ETH	0.00003401
0x54984677b0cb3850...	0x60806040	3592152	11 mins ago	0x4ffF72...0FEE67AD	IN Contract Creation	0 ETH	0.00069221

[Download: CSV Export]

A contract address hosts a smart contract, which is a set of code stored on the blockchain that runs when predetermined conditions are met. Learn more about addresses in our Knowledge Base.

테스트 네트워크에 적용

Sepolia Testnet

Search by Address / Txn Hash / Block / Token

Etherscan

HomeBlockchainTokensNFTsMisc

Contract0x70979003e614b50FD71FEe179CC2576Da5E57611

More

Overview

ETH BALANCE
0.002 ETH

More Info

CONTRACT CREATOR
0x4ffF72...0FEE67AD at txn 0x54984677b0cb3850...

Multi Chain

MULTICHAIN ADDRESSES
1 address found via Blockscan

Transactions

Token Transfers (ERC-20)

Contract

Events

Latest 2 Contract Events

Tip: Logs are used by developers/external UI providers for keeping track of contract actions and for auditing

Txn Hash	Block	Age	Method	Logs
0xc99627dcc7e91e00...	3592169	7 mins ago	0x	[topic0] 0x03b4b447121cfc99f637122d1c595e94ee2fb045709001f086432cf5442ac3f2 Addr → 0x4fff727b33b76e92257af64a58fca56f0fee67ad Num → 10000000000000000
0x91c21e3eddb71a8c...	3592157	10 mins ago	0x	[topic0] 0x03b4b447121cfc99f637122d1c595e94ee2fb045709001f086432cf5442ac3f2 Addr → 0x4fff727b33b76e92257af64a58fca56f0fee67ad Num → 10000000000000000

A contract address hosts a smart contract, which is a set of code stored on the blockchain that runs when predetermined conditions are met. Learn more about addresses in our Knowledge Base.

프로젝트 설명: RandomGame

이 프로젝트에서는 랜덤 게임을 구현합니다. 게임 참가자는 총 10명이 모이면 무작위로 한 명의 우승자가 선정됩니다. 각 참가자는 0.01 이더를 지불하여 게임에 참여하고, 10명이 모두 참여하면 우승자가 0.09 이더를 상금으로 받습니다. 동일한 주소는 한 라운드에서 한 번만 참여할 수 있습니다. 새로운 라운드가 자동으로 시작됩니다.

요구사항

1. 기본 설정

- Solidity 버전은 0.7.0 이상 0.9.0 미만을 사용합니다.
- `RandomGame` 이라는 이름의 스마트 컨트랙트를 작성합니다.

2. 상태 변수

- 게임에 참여한 플레이어들의 주소를 저장하는 배열.
- 게임 참가 비용은 0.01이더
- 각 라운드의 최대 참가자 수 10명
- 우승자가 받는 상금은 90%인 0.09이더

3. 이벤트

- 참가자가 게임에 참여할 때 발생하는 이벤트 출력
- 우승자가 선정될 때 발생하는 이벤트 출력

4. 함수

- `receive` 함수: 이더를 받으면 게임에 참여.
- 매 라운드마다 게임에 참여한 주소가 중복되지 않도록 확인. 한 라운드에 여러번 중복 참여 불가함
- 10명의 참가자가 모이면 무작위로 우승자를 선정하고 상금을 지급.
- 새로운 라운드를 시작하기 위해 게임을 초기화.
- 현재 라운드를 표시해야한다 `getRound()`.
- 현재 스마트 컨트랙트에 쌓인 잔액을 표시한다. 다만 배포한 사람만 실행할 수 있다. `getBalance()`
- 현재 라운드에 들어온 플레이어 수를 표시할 수 있어야 한다. `getPlayerCount()`

5. `random()` 함수는 아래와 같이 생성한다.

```
function random() private view returns (uint) {
    return uint(keccak256(abi.encodePacked(block.prevrandao, block.timestamp, address(this))));
}
```

```

1  // SPDX-License-Identifier: MIT
2  pragma solidity >=0.7.0 <0.9.0;
3
4  contract RandomGame {
5      address public owner;
6      address[] public players;
7      uint public round;
8      uint public constant entryFee = 0.01 ether;
9      uint public constant maxPlayers = 10;
10     uint public constant prizeAmount = 0.09 ether;
11
12     event PlayerEntered(address player);
13     event WinnerSelected(address winner);
14
15     constructor() { 718282 gas 671200 gas
16         owner = msg.sender;
17         round = 1;
18     }
19
20     receive() external payable { undefined gas
21         require(msg.value == entryFee, "Entry fee is 0.01 ether");
22         require(players.length < maxPlayers, "Game is full");
23         require(!isPlayerInGame(msg.sender), "Player already entered");
24         players.push(msg.sender);
25         emit PlayerEntered(msg.sender);
26         if (players.length == maxPlayers) {
27             selectWinner();
28             resetGame();
29         }
30     }
31
32     function isPlayerInGame(address player) internal view returns (bool) {
33         for (uint i = 0; i < players.length; i++) {
34             if (players[i] == player) {
35                 return true;
36             }
37         }
38         return false;
39     }

```



```

41 function selectWinner() private {    ⛽ infinite gas
42     uint index = random() % players.length;
43     address winner = players[index];
44     payable(winner).transfer(prizeAmount);
45     emit WinnerSelected(winner);
46 }
47
48 function resetGame() private {    ⛽ infinite gas
49     delete players;
50     round++;
51 }
52
53 function random() private view returns (uint) {    ⛽ infinite gas
54     return uint(keccak256(abi.encodePacked(block.prevrandao, block.timestamp)));
55 }
56
57 function getPlayers() public view returns (address[] memory) {    ⛽ infinite gas
58     return players;
59 }
60
61 function getPlayerCount() public view returns (uint) {    ⛽ 2511 gas
62     return players.length;
63 }
64
65 function getBalance() public view returns (uint) {    ⛽ 2647 gas
66     require(msg.sender == owner, "Only the owner can check the balance");
67     return address(this).balance;
68 }
69
70 function getRound() public view returns (uint) {    ⛽ 2481 gas
71     return round;
72 }
73 }
74

```

투표 시스템 스마트 컨트랙트 작성

과제 설명

간단한 투표 시스템을 구현하는 스마트 컨트랙트를 작성해야 합니다. 이 과제는 후보자를 등록하고, 투표자에게 투표 권한을 부여하며, 투표를 진행하고, 투표 결과를 확인하는 기능을 포함합니다.

요구사항

1. 컨트랙트 기본 설정

- 스마트 컨트랙트의 이름은 `Voting` 으로 합니다.
- 컨트랙트는 투표 시스템의 소유자, 선거 이름, 투표 시작 및 종료 시간을 저장해야 합니다.
- 0.8.7을 사용한다. `pragma solidity ^0.8.7;`

2. 구조체 정의

- `Candidate`: 후보자의 ID, 이름, 득표 수를 포함합니다.
- `Voter`: 투표자의 권한 상태, 투표 여부, 투표한 후보자 ID를 포함합니다.

3. 컨트랙트 변수

- `owner`: 컨트랙트 소유자의 주소를 저장합니다.
- `electionName`: 선거 이름을 저장합니다.
- `startTime`, `endTime`: 투표 시작 및 종료 시간을 저장합니다.
- `voters`: 투표자의 주소를 키로 하고 `Voter` 구조체를 값으로 가지는 매핑을 사용합니다.
- `candidates`: 후보자의 ID를 키로 하고 `Candidate` 구조체를 값으로 가지는 매핑을 사용합니다.
- `candidatesCount`: 후보자의 수를 저장합니다.
- `totalVotes`: 총 투표 수를 저장합니다.

4. 제한자 (Modifiers)

- `ownerOnly`: 오직 소유자만 해당 함수를 호출할 수 있도록 제한합니다.
- `duringVotingPeriod`: 투표 기간 동안만 접근할 수 있도록 제한합니다.

5. 함수

- 생성자 함수: `electionName`, `startTime`, `endTime` 을 초기화하고 소유자를 설정합니다.
- `addCandidate`: 소유자가 후보자를 추가하는 함수입니다.
- `authorize`: 소유자가 투표자에게 투표 권한을 부여하는 함수입니다.
- `unauthorize`: 소유자가 투표자의 투표 권한을 해제하는 함수입니다.
- `vote`: 투표자가 투표를 수행하는 함수입니다. 투표자는 권한이 부여되어 있어야 하며, 중복 투표가 불가능합니다.
- `end`: 투표를 종료하고 결과를 반환하는 함수입니다.
- `getCandidate`: 특정 후보자의 정보를 조회하는 함수입니다.

6. 이벤트

- `VoteCast`: 투표가 이루어질 때 발생하는 이벤트입니다.
- `CandidateAdded`: 후보자가 추가될 때 발생하는 이벤트입니다.
- `VoterAuthorized`: 투표자에게 권한이 부여될 때 발생하는 이벤트입니다.
- `VoterUnauthorized`: 투표자의 권한이 해제될 때 발생하는 이벤트입니다.

```

1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract Voting {
5      // 후보자(Candidate) 구조체 정의
6      struct Candidate {
7          uint id;          // 후보자 ID
8          string name;      // 후보자 이름
9          uint voteCount;   // 후보자가 받은 투표 수
10     }
11
12     // 투표자(Voter) 구조체 정의
13     struct Voter {
14         bool authorized;   // 투표할 권한이 있는지 여부
15         bool voted;        // 이미 투표했는지 여부
16         uint vote;         // 투표한 후보자 ID
17     }
18
19     address public owner;   // 컨트랙트 소유자 주소
20     string public electionName; // 선거 이름
21     uint public startTime;  // 투표 시작 시간
22     uint public endTime;    // 투표 종료 시간
23
24     mapping(address => Voter) public voters; // 투표자 주소를 키로 하고 Voter 구조체를 값으로 가지는 매핑
25     mapping(uint => Candidate) public candidates; // 후보자 ID를 키로 하고 Candidate 구조체를 값으로 가지는 매핑
26     uint public candidatesCount; // 후보자 수
27     uint public totalVotes;      // 총 투표 수
28
29     // 소유자만 접근할 수 있도록 하는 제한자
30     modifier ownerOnly() {
31         // 여기에 코드를 작성하세요.
32     }
33
34     // 투표 기간 동안만 접근할 수 있도록 하는 제한자
35     modifier duringVotingPeriod() {
36         require(block.timestamp >= startTime && block.timestamp <= endTime, "Voting period has not started or has ended");
37         _;
38     }
39
40     // 생성자: 선거 이름을 설정하고 소유자를 현재 트랜잭션을 보낸 사람으로 설정
41     constructor(string memory _name, uint _votingDuration) {
42         owner = msg.sender;
43         electionName = _name;
44         startTime = block.timestamp;
45         endTime = block.timestamp + _votingDuration;
46     }
47
48     // 후보자를 추가하는 함수, 소유자만 호출 가능
49     function addCandidate(string memory _name) public ownerOnly {
50         // 여기에 코드를 작성하세요
51     }
52
53     // 투표자를 승인하는 함수, 소유자만 호출 가능
54     function authorize(address _person) public ownerOnly {
55         // 여기에 코드를 작성하세요
56     }
57
58     // 투표자를 등록 해제하는 함수, 소유자만 호출 가능
59     function unauthorize(address _person) public ownerOnly {
60         // 여기에 코드를 작성하세요
61     }
62
63     // 투표하는 함수, 투표자는 승인받아야 하고 이미 투표하지 않은 상태여야 함
64     function vote(uint _candidateId) public duringVotingPeriod {
65         // 여기에 코드를 작성하세요
66     }
67
68     // 투표를 종료하고 결과를 반환하는 함수, 소유자만 호출 가능
69     function end() public ownerOnly view returns (uint[] memory) {
70         // 여기에 코드를 작성하세요
71     }
72
73     // 후보자 정보를 조회하는 함수
74     function getCandidate(uint _candidateId) public view returns (string memory, uint) {
75         // 여기에 코드를 작성하세요
76     }
77
78     // 이벤트 로그 정의
79     event VoteCast(address voter, uint candidateId);
80     event CandidateAdded(uint candidateId, string name);
81     event VoterAuthorized(address voter);
82     event VoterUnauthorized(address voter);
83 }

```



```
// 투표를 종료하고 결과를 반환하는 함수, 소유자만 호출 가능
function end() public ownerOnly view returns (uint[] memory) {
    uint[] memory result = new uint[](candidatesCount); // 후보자 수 크기의 배열 생성
    for (uint i = 1; i <= candidatesCount; i++) {
        result[i-1] = candidates[i].voteCount; // 각 후보자의 득표수 배열에 저장
    }
    return result; // 득표수 배열 반환
}
```

솔리디티에서 `mapping` 은 연관 배열 또는 해시와 같은 키-값 데이터 구조를 나타냅니다. 이는 키를 사용하여 값을 검색할 수 있도록 허용하는 구조입니다.

`mapping` 을 선언하려면, `mapping` 키워드를 사용하고, 괄호 안에 키 타입과 값 타입을 지정하십시오. 예를 들어, 주소를 키로 사용하고, 정수를 값으로 사용하는 맵핑을 선언하려면 다음과 같이 작성할 수 있습니다.

```
mapping(address => uint) public balances;
```

위의 예제에서 `balances` 라는 이름의 맵핑을 선언했습니다. 이 맵핑은 주소를 키로 사용하고, 이 키에 대응하는 값으로 `uint`(부호 없는 정수)를 사용합니다. `public` 키워드를 사용하면 솔리디티가 자동으로 getter 함수를 생성합니다.

맵핑에 값을 할당하려면, 대괄호 안에 키를 넣고, 이 키에 대응하는 값을 할당합니다.

```
balances[msg.sender] = 100;
```

마찬가지로, 맵핑에서 값을 검색하려면 대괄호 안에 키를 넣습니다.

```
uint balance = balances[msg.sender];
```

맵핑은 기본적으로 모든 가능한 키에 대해 값을 0으로 초기화합니다. 따라서 맵핑 내에 특정 키가 존재하는지 확인하는 방법은 없습니다. 이를 해결하기 위해 맵핑과 함께 사용할 수 있는 "기록" 배열을 사용하는 것이 일반적입니다.

또한, 맵핑은 다른 맵핑 또는 동적 배열을 값으로 사용할 수 있습니다. 이를 이용하면 복잡한 데이터 구조를 만들 수 있습니다.

```
// 주소를 키로 사용하고, 다른 맵핑을 값으로 사용하는 맵핑
mapping(address => mapping(uint => bool)) public nestedMapping;

// 값을 설정
nestedMapping[msg.sender][1] = true;

// 값을 검색
bool value = nestedMapping[msg.sender][1];
```

`mapping` 을 이용하여 주소를 키로 하고, 구조체를 값으로 사용하는 방법은 다음과 같습니다:

먼저, 구조체를 선언합니다:

```
struct User {  
    uint id;  
    string name;  
    bool isRegistered;  
}
```

그 다음, 주소를 키로 하고 `User` 구조체를 값으로 하는 `mapping` 을 선언합니다:

```
mapping(address => User) public users;
```

이제 특정 주소에 대응하는 `User` 를 생성하거나 수정할 수 있습니다:

```
users[msg.sender] = User(1, "Alice", true);
```

이 `mapping` 에서 특정 주소에 대응하는 `User` 를 가져올 수도 있습니다:

```
User memory user = users[msg.sender];
```

이렇게 하면, 각 주소에 대응하는 `User` 정보를 쉽게 저장하고 검색할 수 있습니다. 하지만, `mapping` 은 기본적으로 모든 가능한 키에 대해 값을 0으로 초기화하므로 특정 주소에 대응하는 `User` 가 실제로 설정되었는지 아닌지를 확인하기 위해서는 추가적인 로직이 필요합니다. 이를 위해, 보통 특정 주소가 등록되었는지를 확인하는 별도의 `mapping` 을 함께 사용합니다:

```
mapping(address => bool) public isUser;
```

`mapping` 에서 없는 값을 접근하려고 하면 기본적으로 0 또는 해당 타입의 초기값이 반환됩니다. 이는 `mapping` 이 모든 가능한 키에 대해 값을 0으로 초기화하기 때문입니다. 아래에 몇 가지 예를 들어보겠습니다.

1. 정수형을 값으로 가지는 `mapping` 에서 없는 키를 접근하는 경우:

```
mapping(address => uint) public balances;  
uint balance = balances[address(0)]; // 0이 반환됩니다.
```

1. 주소를 값으로 가지는 `mapping` 에서 없는 키를 접근하는 경우:

```
mapping(uint => address) public users;  
address user = users[1]; // 0x0000000000000000000000000000000000000000000000000000000000000000 (주소의 초기  
값)이 반환됩니다.
```

1. bool 타입을 값으로 가지는 `mapping` 에서 없는 키를 접근하는 경우:

```
mapping(address => bool) public isVoter;  
bool voter = isVoter[address(0)]; // false(불리언의 초기값)가 반환됩니다.
```

1. 구조체를 값으로 가지는 `mapping` 에서 없는 키를 접근하는 경우:

```
struct User {  
    uint id;  
    string name;  
    bool isRegistered;  
}  
mapping(address => User) public users;  
User memory user = users[address(0)]; // 모든 필드가 초기값(0, 빈 문자열, false)으로 설  
정된 User가 반환됩니다.
```

이렇게 `mapping` 에서 없는 값을 접근하면 초기값이 반환되므로, 특정 키가 `mapping` 에 존재하는지 아닌지를 확인하기 위해서는 별도의 로직이 필요합니다. 일반적으로 이를 위해 특정 키가 등록되었는지를 확인하는 별도의 `mapping` 을 함께 사용합니다.